

Programovacie jazyky

June 5, 2026

1 Zoznamy, n-tice, ADT, GADT, abstraktné dátové typy a moduly

1.1 Zoznamy

Zoznam je v Haskellí homogénna dátová štruktúra, teda všetky jeho prvky musia mať rovnaký typ. Typ zoznamu prvkov typu `a` zapisujeme ako `[a]`.

```
[1,2,3]      :: [Int]
['a','b']    :: [Char]
[[1,2],[3]]  :: [[Int]]
```

Zoznam môže byť prázdny:

```
[] :: [a]
```

alebo vytvorený pomocou konštruktora `(:)`, ktorý pridá prvok na začiatok zoznamu:

```
1 : [2,3] == [1,2,3]
1 : 2 : 3 : [] == [1,2,3]
```

Konštruktor `(:)` má typ:

```
(:) :: a -> [a] -> [a]
```

Zoznam je teda rekurzívny typ: buď je prázdny, alebo pozostáva z hlavy a zvyšku zoznamu.

1.1.1 Práca so zoznamami

Typické funkcie nad zoznamami sú:

```
map      :: (a -> b) -> [a] -> [b]
filter   :: (a -> Bool) -> [a] -> [a]
length   :: [a] -> Int
sum       :: Num a => [a] -> a
```

```

product :: Num a => [a] -> a
take    :: Int -> [a] -> [a]
drop    :: Int -> [a] -> [a]
concat  :: [[a]] -> [a]

```

Príklady:

```

map (*2) [1,2,3]      == [2,4,6]
filter odd [1,2,3,4]  == [1,3]
take 3 [1,2,3,4,5]    == [1,2,3]
drop 3 [1,2,3,4,5]    == [4,5]
concat [[1,2],[3,4]]  == [1,2,3,4]

```

Funkcia `filter` sa dá chápať aj cez zoznamovú komprehéziu:

```
filter p xs = [ x | x <- xs, p x ]
```

1.1.2 Pattern matching nad zoznamami

Zoznamy sa často spracúvajú pomocou vzorov:

```

sumList :: Num a => [a] -> a
sumList []      = 0
sumList (x:xs) = x + sumList xs

```

Prvý prípad rieši prázdny zoznam. Druhý prípad rozloží neprázdny zoznam na prvý prvok `x` a zvyšok `xs`.

1.2 N-tice

N-tica je dátová štruktúra pevnej dĺžky, ktorá môže obsahovať hodnoty rôznych typov. Na rozdiel od zoznamov teda nemusí byť homogénna.

```

(1, "ahoj")      :: (Int, String)
(True, 3, 'x')    :: (Bool, Int, Char)

```

Dvojica má typ:

```
(a, b)
```

Trojica má typ:

```
(a, b, c)
```

Pre dvojice existujú funkcie:

```

fst :: (a,b) -> a
snd :: (a,b) -> b

```

Príklad:

```
fst (1, "x") == 1
snd (1, "x") == "x"
```

N-tice sú vhodné vtedy, keď potrebujeme zoskupiť pevný počet hodnôt. Ak však hodnoty majú pomenovaný význam, často je lepšie použiť vlastný dátový typ alebo záznam.

1.3 Algebraické dátové typy

Algebraické dátové typy, skrátene ADT v zmysle *algebraic data types*, umožňujú definovať nové typy pomocou konštruktorov. V Haskellu sa definujú kľúčovým slovom `data`.

Rozlišujeme najmä:

- produktové typy,
- sumové typy,
- rekurzívne typy,
- parametrické typy.

1.3.1 Produktové typy

Produktový typ obsahuje viacero zložiek naraz. Príkladom je typ `Student`:

```
data Student = MkSt String String Int String
```

Tu `Student` je nový typ a `MkSt` je dátový konštruktor. Konštruktor má typ:

```
MkSt :: String -> String -> Int -> String -> Student
```

Teda z viacerých hodnôt vytvorí jednu hodnotu typu `Student`.

Príklad hodnoty:

```
st :: Student
st = MkSt "Janko" "Hrasko" 4 "sikovny"
```

Produktový typ zodpovedá logickému “a zároveň”: študent má meno, priezvisko, ročník aj poznámku.

1.3.2 Sumové typy

Sumový typ vyjadruje alternatívu. Hodnota môže byť vytvorená jedným z viacerých konštruktorov.

Príklad:

```
data Bool = False | True
```

Hodnota typu `Bool` je buď `False`, alebo `True`. Takýto špeciálny prípad sa nazýva aj vymenovaný typ.

Další príklad:

```
data Maybe a = Nothing | Just a
```

Typ `Maybe a` reprezentuje hodnotu, ktorá môže chýbať. Buď je to `Nothing`, alebo `Just x`, kde `x` má typ `a`.

Konštruktory majú typy:

```
Nothing :: Maybe a
Just     :: a -> Maybe a
```

Príklad použitia:

```
safeHead :: [a] -> Maybe a
safeHead []      = Nothing
safeHead (x:_)   = Just x
```

1.3.3 Rekurzívne algebraické dátové typy

Dátový typ môže byť definovaný rekurzívne, teda môže odkazovať sám na seba.

Príklad binárneho stromu:

```
data Tree a = Leaf a
            | Node (Tree a) (Tree a)
```

Hodnota typu `Tree a` je buď list s hodnotou typu `a`, alebo vnútorný uzol s dvoma podstromami.

Príklad funkcie nad stromom:

```
size :: Tree a -> Int
size (Leaf _)      = 1
size (Node l r)    = size l + size r
```

1.3.4 Parametrické dátové typy

Dátový typ môže mať typový parameter. Napríklad:

```
data Maybe a = Nothing | Just a
```

Tu `Maybe` nie je konkrétny typ, ale typový konštruktor. Až `Maybe Int`, `Maybe Bool` alebo `Maybe String` sú konkrétne typy.

Druh typového konštruktora `Maybe` je:

```
Maybe :: * -> *
```

To znamená, že `Maybe` berie jeden konkrétny typ a vytvorí nový konkrétny typ.

1.4 GADT

GADT znamená *Generalized Algebraic Data Type*, teda zovšeobecnený algebraický dátový typ.

Pri obyčajnom ADT majú všetky dátové konštruktory rovnaký výsledný typ. Napríklad:

```
data Expr = I Int
          | B Bool
          | Add Expr Expr
          | Lt Expr Expr
```

Problém je, že tento typ dovolí vytvoriť aj sémanticky nesprávny výraz:

```
Add (I 5) (B True)
```

Syntakticky je to správne, ale sémanticky nie, pretože sčítavame celé číslo a boolovskú hodnotu. Potom musí funkcia `eval` ručne kontrolovať chyby, napríklad cez `Maybe`.

GADT umožňujú presnejšie určiť návratový typ každého dátového konštruktora.

```
data Expr a where
  I    :: Int -> Expr Int
  B    :: Bool -> Expr Bool
  Add  :: Expr Int -> Expr Int -> Expr Int
  Mul  :: Expr Int -> Expr Int -> Expr Int
  Lt   :: Expr Int -> Expr Int -> Expr Bool
```

Tu typový parameter `a` vyjadruje typ hodnoty, na ktorú sa výraz vyhodnotí. Konštruktor `I` vytvára výraz typu:

```
Expr Int
```

Konštruktor `B` vytvára výraz typu:

`Expr Bool`

Konštruktor `Lt` berie dva celočíselné výrazy a vracia boolovský výraz:

`Lt :: Expr Int -> Expr Int -> Expr Bool`

Vďaka tomu už typový systém odmietne nesprávne výrazy:

`Add (I 5) (B True)`

pretože `Add` očakáva dva argumenty typu `Expr Int`, ale `B True` má typ `Expr Bool`.

Výhodou GADT je, že funkciu `eval` môžeme napísať typovo presne:

```
eval :: Expr a -> a
eval (I n)      = n
eval (B b)      = b
eval (Add x y)   = eval x + eval y
eval (Mul x y)   = eval x * eval y
eval (Lt x y)    = eval x < eval y
```

Typ výsledku závisí od typu výrazu:

```
eval (Add (I 1) (I 2)) :: Int
eval (Lt (I 1) (I 2))  :: Bool
```

Hlavná myšlienka GADT je teda tá, že jednotlivé dátové konštruktory môžu vracať špecializované inštalácie parametrického typu, napríklad `Expr Int` alebo `Expr Bool`, nie iba všeobecné `Expr a`.

1.5 Abstraktné dátové typy

Abstraktný dátový typ, skrátene ADT v zmysle *abstract data type*, je dátový typ, pri ktorom používateľ pozná rozhranie, ale nepozná alebo nemôže priamo používať implementačné detaily.

Dôležité je rozlišovať:

- *algebraic data type* – spôsob definície dátového typu pomocou konštruktorov,
- *abstract data type* – spôsob zapuzdrenia dátového typu cez verejné rozhranie.

V Haskellu sa abstraktné dátové typy vytvárajú pomocou modulov: exportujeme typ, ale neexportujeme jeho dátové konštruktory.

Príklad zásobníka:

```
module StackADT (Stack, empty, isEmpty, push, pop) where

data Stack a = Empty | Push a (Stack a)
```

```

empty :: Stack a
empty = Empty

isEmpty :: Stack a -> Bool
isEmpty Empty = True
isEmpty _     = False

push :: a -> Stack a -> Stack a
push x s = Push x s

pop :: Stack a -> (a, Stack a)
pop Empty      = error "Stack: popping from empty stack"
pop (Push x s) = (x, s)

```

V exportnom zozname je uvedený typ `Stack`, ale nie jeho konštruktory `Empty` a `Push`. Používateľ teda nemôže priamo vytvoriť alebo rozobrať hodnotu typu `Stack a`; musí používať funkcie `empty`, `isEmpty`, `push` a `pop`.

Výhody abstraktných dátových typov:

- ukrytie implementačných detailov,
- možnosť meniť implementáciu bez zmeny verejného rozhrania,
- ochrana invariantov dátovej štruktúry,
- jasné oddelenie rozhrania od implementácie.

1.6 Moduly

Modul je zoskupenie súvisiacich funkcií, typov, typových tried, konštánt a ďalších definícií. Program sa skladá z modulov. Moduly môžu importovať iné moduly a používať ich exportované definície.

Základná syntax modulu je:

```
module MenoModulu (exportovaneNazvy) where
```

Príklad:

```

module MyList (myLength, myMap) where

myLength :: [a] -> Int
myLength []      = 0
myLength (_:xs) = 1 + myLength xs

myMap :: (a -> b) -> [a] -> [b]
myMap _ []      = []
myMap f (x:xs) = f x : myMap f xs

```

Ak modul nemá explicitný exportný zoznam, exportujú sa všetky definície:

```
module MyList where
```

Ak súbor nezačína deklaráciou modulu, predpokladá sa hlavný modul:

```
module Main(main) where
```

1.6.1 Export typov a konštruktorov

Pri exporte dátových typov môžeme rozhodnúť, či exportujeme aj ich konštruktory.

Export typu aj všetkých konštruktorov:

```
module Maybe (Maybe(..)) where
```

Export typu a konkrétnych konštruktorov:

```
module Maybe (Maybe(Nothing, Just)) where
```

Export iba typu bez konštruktorov:

```
module StackADT (Stack, empty, push, pop) where
```

Posledný spôsob je základom pre tvorbu abstraktných dátových typov.

1.6.2 Import modulov

Moduly sa importujú pomocou `import`:

```
import Data.List
```

Môžeme importovať iba vybrané názvy:

```
import Data.List (sort, nub)
```

Môžeme niektoré názvy skryť:

```
import Data.List hiding (sort)
```

Alebo môžeme importovať modul kvalifikovane:

```
import qualified Data.Map as Map
```

Potom používame názvy s prefixom:

```
Map.lookup key mapa
```

Výhody modulov:

- kontrola menného priestoru,
- menší počet kolízií mien,
- lepšia znovupoužitelnosť,
- jednoduchšie rozdelenie práce,
- ukrytie implementačných detailov,
- možnosť vytvárať abstraktné dátové typy.

1.7 Porovnanie pojmov

Pojem	Význam
Zoznam	Homogénna sekvencia prvkov typu [a].
N-tica	Pevný počet hodnôt, potenciálne rôznych typov.
Algebraický dátový typ	Typ definovaný pomocou dátových konštruktorov.
Produktový typ	Hodnota obsahuje viacero zložiek naraz.
Sumový typ	Hodnota je jednou z viacerých alternatív.
GADT	Zovšeobecnený ADT, kde konštruktory môžu mať presnejší návratový typ.
Abstraktný dátový typ	Typ so skrytými implementačnými detailmi a verejným rozhraním.
Modul	Jednotka organizácie kódu a mechanizmus exportu/importu mien.

1.8 Krátke zhrnutie

Zoznamy sú homogénne rekurzívne štruktúry, zatiaľ čo n-tice majú pevnú dĺžku a môžu obsahovať rôzne typy. Algebraické dátové typy umožňujú definovať nové typy pomocou dátových konštruktorov, pričom produktové typy spájajú viacero hodnôt a sumové typy reprezentujú alternatívy. GADT sú zovšeobecnením ADT, ktoré umožňuje presnejšie typovať dátové konštruktory. Abstraktné dátové typy skrývajú implementáciu a poskytujú verejné rozhranie. V Haskellu sa abstrakcia dosahuje pomocou modulov a exportných zoznamov.

2 Pattern matching, kontrola úplnosti, chvostová rekurzia, akumulátor a typy ranku-n

2.1 Pattern matching

Pattern matching, po slovensky porovnávanie so vzorom, je mechanizmus, ktorý umožňuje definovať funkcie podľa tvaru vstupných dát. Namiesto toho, aby sme hodnotu explicitne testovali pomocou `if`, môžeme priamo uviesť vzory, ktorým sa argumenty funkcie majú prispôbiť.

Príklad definície faktoriálu pomocou porovnávania so vzorom:

```
factorial :: (Eq a, Num a) => a -> a
factorial 0 = 1
factorial 1 = 1
factorial n = n * factorial (n - 1)
```

Význam je nasledovný: ak je argument 0, výsledok je 1; ak je argument 1, výsledok je tiež 1; inak sa použije posledný vzor `n`.

Pattern matching sa vyhodnocuje zhora nadol. Použije sa prvý vzor, ktorý sa zhoduje so vstupom.

2.2 Vzory nad zoznamami

Typickým príkladom pattern matchingu je spracovanie zoznamov.

```
sumList :: Num a => [a] -> a
sumList []      = 0
sumList (x:xs) = x + sumList xs
```

Vzory majú tento význam:

- [] označuje prázdny zoznam,
- (x:xs) označuje neprázdny zoznam, kde **x** je prvý prvok a **xs** je zvyšok zoznamu.

Podobne môžeme definovať funkciu, ktorá bezpečne vráti prvý prvok zoznamu:

```
safeHead :: [a] -> Maybe a
safeHead []      = Nothing
safeHead (x:_) = Just x
```

Použitie typu `Maybe a` zabezpečí, že funkcia je totálna: pre každý vstup vráti definovaný výsledok.

2.3 Wildcard vzor

Znak `_` je anonymný vzor, ktorý sa zhoduje s ľubovoľnou hodnotou, ale túto hodnotu nepomenuje.

```
const42 :: a -> Int
const42 _ = 42
```

Pri zoznamoch:

```
second :: [a] -> Maybe a
second (_,x:_) = Just x
second _       = Nothing
```

Druhý riadok zachytí všetky prípady, ktoré neboli pokryté prvým vzorom.

2.4 As-pattern

As-pattern umožňuje pomenovať celý vstup a zároveň ho rozložiť na časti. Zapisuje sa pomocou `@`.

```
rastie :: Ord a => [a] -> Bool
rastie []           = True
rastie [_]          = True
rastie xs@(x:y:ys) = x < y && rastie (y:ys)
```

Premenná **xs** označuje celý zoznam, zatiaľ čo **x**, **y** a **ys** označujú jeho časti.

2.5 Guards

Pattern matching možno kombinovať s podmienkami, ktoré sa nazývajú guards.

```
factorial :: (Ord a, Num a) => a -> a
factorial n
  | n < 2      = 1
  | otherwise = n * factorial (n - 1)
```

`otherwise` nie je špeciálne kľúčové slovo, ale konštanta:

```
otherwise :: Bool
otherwise = True
```

Preto sa používa ako posledná podmienka, ktorá zachytí všetky zvyšné prípady.

2.6 Case výraz

Pattern matching vo funkciách je možné prepísať pomocou výrazu `case`.

Všeobecná syntax:

```
case vyraz of
  vzor1 -> vysledok1
  vzor2 -> vysledok2
  _      -> inyVysledok
```

Príklad:

```
safeHead :: [a] -> Maybe a
safeHead xs =
  case xs of
    []      -> Nothing
    (x:_)   -> Just x
```

Definícia funkcie pomocou viacerých rovníc je teda sémanticky podobná definícii pomocou `case`.

2.7 Kontrola úplnosti pattern matchingu

Kontrola úplnosti zisťuje, či sú vo funkcii pokryté všetky možné tvary vstupných hodnôt. Ak niektorý prípad chýba, funkcia je čiastočná a môže spôsobiť runtime chybu.

Príklad neúplnej definície:

```
neg :: Bool -> Bool
neg True = False
```

Táto funkcia nepokrýva vstup `False`. Kompilátor GHC preto vypíše varovanie podobné:

```
warning: [-Wincomplete-patterns]
Pattern match(es) are non-exhaustive
Patterns not matched: False
```

Ak zavoláme:

```
neg False
```

program skončí chybou:

```
*** Exception: Non-exhaustive patterns in function neg
```

Správna úplná definícia je:

```
neg :: Bool -> Bool
neg True  = False
neg False = True
```

2.8 Čiastočné a totálne funkcie

Funkcia je totálna, ak pre každý vstup zo svojho definičného oboru vráti výsledok. Funkcia je čiastočná, ak pre niektoré vstupy skončí chybou alebo nie je definovaná.

Príklad čiastočnej funkcie:

```
prvy :: [a] -> a
prvy (x:_) = x
```

Problém je, že nie je definovaná pre prázdny zoznam `[]`.
Explicitne čiastočná verzia:

```
prvy :: [a] -> a
prvy (x:_) = x
prvy []    = error "Prázdny zoznam"
```

Lepšie riešenie je totálna funkcia:

```
prvy :: [a] -> Maybe a
prvy []    = Nothing
prvy (x:_) = Just x
```

Na skúške je dobré zdôrazniť, že varovania o neúplných vzoroch netreba ignorovať. V Haskellí je bežné preferovať totálne funkcie pred čiastočnými.

2.9 Príklad zložitejšej kontroly úplnosti

Uvažujme funkciu, ktorá zisťuje, či má zoznam dĺžku deliteľnú tromi:

```
troj :: [a] -> Bool
troj []          = True
troj [_]         = False
troj (_:_:_:xs) = troj xs
```

Táto definícia nie je úplná, pretože chýba prípad zoznamu s dvoma prvkami:

```
[_, _]
```

Správna verzia:

```
troj :: [a] -> Bool
troj []          = True
troj (_:_:_:xs) = troj xs
troj _          = False
```

Posledný vzor `_` zachytí všetky zvyšné prípady. Treba ho však používať opatrne: ak ho dáme príliš skoro, môže zakryť ďalšie špecifické prípady.

2.10 Poradie vzorov

Poradie vzorov je dôležité, pretože sa skúšajú zhora nadol.

Príklad:

```
f :: [a] -> String
f [] = "prazdny"
f _  = "neprazdny"
```

Táto funkcia funguje správne. Ak by sme však poradie zamenili:

```
f :: [a] -> String
f _  = "nejaky zoznam"
f [] = "prazdny"
```

druhý riadok sa nikdy nepoužije, pretože vzor `_` zachytí všetky vstupy.

2.11 Chvostová rekúzia

Rekurzívna funkcia je chvostovo rekurzívna, ak je rekurzívne volanie poslednou operáciou, ktorú funkcia vykoná. To znamená, že po návrate z rekurzívneho volania už netreba robiť žiadnu ďalšiu prácu.

Nechvostovo rekurzívny faktoriál:

```
factorial :: Integer -> Integer
factorial 0 = 1
factorial n = n * factorial (n - 1)
```

Táto funkcia nie je chvostovo rekurzívna, pretože po návrate z `factorial (n - 1)` treba ešte vynásobiť výsledok číslom `n`.

Chvostovo rekurzívna verzia používa akumulátor:

```
factorialTR :: Integer -> Integer
factorialTR n = fact n 1
  where
    fact 0 acc = acc
    fact n acc = fact (n - 1) (n * acc)
```

Rekurzívne volanie:

```
fact (n - 1) (n * acc)
```

je posledná operácia vo funkcii. Výsledok sa postupne ukladá do akumulátora `acc`.

2.12 Akumulátor

Akumulátor je pomocný parameter, v ktorom si rekurzívna funkcia nesie priebežný výsledok výpočtu. Používa sa najmä pri transformácii nechvostovej rekurzie na chvostovú.

Príklad výpočtu dĺžky zoznamu:

```
lengthTR :: [a] -> Int
lengthTR xs = length' xs 0
  where
    length' [] acc = acc
    length' (_:xs) acc = length' xs (acc + 1)
```

Akumulátor `acc` obsahuje počet prvkov, ktoré už boli spracované.

Výpočet napríklad pre `[a,b,c]`:

```
length' [a,b,c] 0
length' [b,c]    1
length' [c]      2
length' []       3
3
```

2.13 Chvostovo rekurzívne reverse

Bežná definícia `reverse` je nechvostovo rekurzívna:

```
reverse :: [a] -> [a]
reverse [] = []
reverse (x:xs) = reverse xs ++ [x]
```

Problém je, že po rekurzívnom volaní `reverse xs` sa ešte vykoná operácia `++ [x]`. Navyše, operátor `++` musí prechádzať ľavý argument, takže táto verzia je neefektívna.

Chvostovo rekurzívna verzia:

```
reverseTR :: [a] -> [a]
reverseTR xs = rev [] xs
  where
    rev acc []      = acc
    rev acc (x:xs) = rev (x:acc) xs
```

Akumulátor `acc` obsahuje už otočenú časť zoznamu.

Priebeh výpočtu:

```
rev []      [1,2,3]
rev [1]     [2,3]
rev [2,1]   [3]
rev [3,2,1] []
[3,2,1]
```

2.14 Viac akumulátorov

Niekedy jeden akumulátor nestačí. Napríklad pri chvostovo rekurzívnom výpočte Fibonacciho čísel si môžeme nieš dve predchádzajúce hodnoty.

```
fibTR :: Integer -> Integer
fibTR n = fib' 0 1 n
  where
    fib' f1 _ 0 = f1
    fib' f1 f2 n = fib' f2 (f1 + f2) (n - 1)
```

Tu `f1` a `f2` predstavujú dve po sebe idúce Fibonacciho hodnoty. V každom kroku sa posúvajú ďalej.

2.15 Význam chvostovej rekurzcie

Chvostová rekurzcia je dôležitá preto, že v mnohých jazykoch ju kompilátor vie optimalizovať podobne ako cyklus. Namiesto toho, aby sa na zásobník ukladali všetky čakajúce operácie, stačí aktualizovať argumenty pomocnej funkcie.

V Haskellí treba byť opatrný, pretože kvôli lenivému vyhodnocovaniu môže akumulátor vytvárať reťaz nevyhodnotených výrazov. V praxi sa preto často používajú striktnéjšie verzie funkcií, napríklad `foldl'` namiesto `foldl`.

2.16 Typy ranku-n

Typy ranku-n, po anglicky *Rank-N types*, sú typy, v ktorých sa univerzálny kvantifikátor `forall` môže nachádzať aj na ľavej strane funkčnej šípky `->`, teda vo vnútri typu argumentu funkcie.

Bežné polymorfické typy v Haskellí sú ranku 1. Napríklad:

```
id :: forall a. a -> a
```

alebo:

```
const :: forall a b. a -> b -> a
```

Pri typoch ranku 1 môžeme všetky kvantifikátory `forall` presunúť na začiatok typu.

2.17 Motivácia pre Rank-N types

Majme polymorfickú funkciu:

```
f :: a -> (a, Int)
f x = (x, 1)
```

Chceli by sme napísať funkciu, ktorá dostane polymorfickú funkciu `f` a použije ju na argumenty rôznych typov:

```
g f x y = (f x, f y)
```

Chceli by sme napríklad:

```
g f True 'a'
```

s výsledkom:

```
((True,1),('a',1))
```

Problém je, že bez špeciálnej typovej anotácie Haskell odvodí typ, kde `x` a `y` musia mať rovnaký typ:

```
g :: (a -> b) -> a -> a -> (b, b)
```

To nestačí, pretože `True` má typ `Bool`, zatiaľ čo `'a'` má typ `Char`.

Potrebujeme povedať, že argument `f` je sám polymorfická funkcia, ktorú možno použiť pre ľubovoľný typ `a`.

2.18 Príklad typu ranku 2

Správna typová anotácia je:

```
{-# LANGUAGE Rank2Types #-}
```

```
f :: a -> (a, Int)
f x = (x, 1)
```

```
g :: (forall a. a -> (a, Int))
    -> b
    -> c
    -> ((b, Int), (c, Int))
g f x y = (f x, f y)
```


Teraz možno písať:

```
v :: ((Bool, Int), (Char, Int))
v = g f True 'a'
```

Tu je dôležité, že `forall a. a -> (a, Int)` sa nachádza ako typ argumentu funkcie `g`. Teda `g` dostáva ako argument polymorfickú funkciu.

Tento typ je ranku 2, pretože funkcia `g` má argument, ktorého typ je polymorfický.

2.19 Intuícia ranku

Neformálne:

- rank 1: `forall` je iba na začiatku celého typu,
- rank 2: funkcia berie ako argument polymorfickú funkciu ranku 1,
- rank n: funkcia má aspoň jeden argument ranku n-1, ale žiadny argument vyššieho ranku.

Príklad ranku 1:

```
id :: forall a. a -> a
```

Príklad ranku 2:

```
applyPoly :: (forall a. a -> a) -> (Bool, Char)
applyPoly f = (f True, f 'x')
```

Funkcia `applyPoly` vyžaduje, aby jej argument `f` fungoval pre každý typ `a`. Preto ho môže použiť raz ako funkciu `Bool -> Bool` a raz ako funkciu `Char -> Char`.

2.20 Rozdiel medzi dvoma podobnými typmi

Tieto dva typy vyzerajú podobne, ale znamenajú niečo iné:

```
g1 :: forall a b c. (a -> (a, Int)) -> b -> c -> ((b, Int), (c, Int))
```

a:

```
g2 :: forall b c. (forall a. a -> (a, Int))
  -> b -> c -> ((b, Int), (c, Int))
```

V prvom type sa `a` zvolí raz pre celé volanie. Funkcia teda nie je dostatočne polymorfná na to, aby sme ju použili na `b` aj `c`, ak sú to rôzne typy.

V druhom type je samotný argument polymorfický. Funkciu možno použiť nezávisle pre rôzne typy.

2.21 Prečo treba typové anotácie

Haskell používa Hindleyho-Milnerov typový systém, ktorý vie automaticky odvodiť najmä typy ranku 1. Pre typy vyššieho ranku je typová inferencia všeobecne nerozhodnuteľná. Preto pri Rank-N types potrebujeme explicitné typové anotácie.

V GHC treba zapnúť rozšírenie:

```
{-# LANGUAGE RankNTypes #-}
```

alebo starší názov:

```
{-# LANGUAGE Rank2Types #-}
```

2.22 Rank-N types a bezpečnosť

Typy vyššieho ranku sa používajú aj na zabezpečenie toho, aby funkcia nemohla “vyniesť” alebo zneužiť konkrétny typ vstupnej hodnoty.

Príklad:

```
len :: forall a. [a] -> Int
len = length
```

```
total :: [Int] -> Int
total = sum
```

Funkcia `len` je skutočne polymorfická: funguje pre zoznam ľubovoľných prvkov a nemôže sa pozerať na konkrétnu štruktúru prvkov. Funkcia `total` funguje iba pre `[Int]`.

Porovnajme:

```
f :: forall a. [a] -> ([a] -> Int) -> Int
f list h = h list
```

Tu funkcia `h` má typ `[a] -> Int`, kde `a` je rovnaké ako typ prvkov vstupného zoznamu. Ak je teda zoznam typu `[Int]`, môžeme použiť aj `sum`.

Bezpečnejšia verzia:

```
g :: forall a. [a] -> (forall a. [a] -> Int) -> Int
g list h = h list
```

Tu musí byť `h` polymorfická pre každý typ `a`. Preto možno použiť `len`, ale nie `total`:

```
g [3..10] len    -- OK
g [3..10] total  -- typova chyba
```

Dôvod je, že `total` vie pracovať iba so zoznamom celých čísel, zatiaľ čo `g` vyžaduje funkciu použiteľnú pre ľubovoľný typ prvkov.

2.23 Krátke zhrnutie

Pattern matching umožňuje definovať funkcie podľa tvaru vstupných dát. Vzory sa skúšajú zhora nadol a ak žiadny vzor nevyhovuje, vznikne runtime chyba. Preto je dôležitá kontrola úplnosti vzorov a preferovanie totálnych funkcií. Chvostová rekúzia je taká rekúzia, kde je rekurzívne volanie poslednou operáciou funkcie. Na jej dosiahnutie sa často používa akumulátor, ktorý nesie priebežný výsledok. Typy ranku- n umožňujú, aby argument funkcie bol sám polymorfický; typicky vyžadujú explicitné typové anotácie a v GHC rozšírenie `RankNTypes`.

3 Striktné, odložené a lenivé vyhodnocovanie, thunks, foldl vs. foldl', mutable a immutable dátové štruktúry

3.1 Striktné vyhodnocovanie

Striktné vyhodnocovanie, po anglicky *strict evaluation* alebo *eager evaluation*, znamená, že argumenty funkcie sa vyhodnotia ešte pred zavolaním funkcie.

Táto stratégia sa nazýva aj *call-by-value*.

Príklad v striktnom jazyku:

```
f (g x)
```

Najprv sa vyhodnotí `g x` a až potom sa výsledok odovzdá funkcii `f`.

Striktné vyhodnocovanie používajú napríklad jazyky ako C, Java, Racket alebo väčšina bežných imperatívnych jazykov. V materiáloch sa uvádza, že Racket má striktné vyhodnocovanie argumentov.

3.1.1 Výhody striktného vyhodnocovania

- jednoduchší model vyhodnocovania,
- predvídateľnejšie poradie efektov,
- často jednoduchšia implementácia,
- vhodné pre imperatívne jazyky s vedľajšími efektmi.

3.1.2 Nevýhody striktného vyhodnocovania

Nevýhodou je, že sa môže vyhodnotiť aj argument, ktorý funkcia vôbec nepoužije.

Príklad:

```
const 1 expensiveComputation
```

Ak je jazyk striktný, `expensiveComputation` sa vyhodnotí, hoci funkcia `const` druhý argument nepotrebuje.

3.2 Špeciálne formy a striktnosť

Aj v striktných jazykoch existujú konštrukcie, ktoré nemôžu byť obyčajné funkcie, pretože by sa ich argumenty vyhodnotili príliš skoro.

Typickým príkladom je `if`. V striktnom jazyku sa pri výraze:

```
if p then e1 else e2
```

musí najprv vyhodnotiť podmienka `p`, ale potom sa vyhodnotí iba jedna vetva: buď `e1`, alebo `e2`. Druhá vetva sa nesmie vyhodnotiť, pretože môže obsahovať chybu, nekonečný výpočet alebo nežiaduci vedľajší efekt.

V Racketovom príklade:

```
(define (bad-if p t e) (if p t e))
```

```
(define (fact n)
  (bad-if (< n 1)
    1
    (* n (fact (sub1 n)))))
```

sa `bad-if` správa ako obyčajná funkcia. Preto sa pred jej zavolaním vyhodnotia všetky argumenty, vrátane rekurzívneho volania. Výsledkom je, že sa faktoriál zacyklí aj pre vstup 0. Materiál zdôrazňuje, že výmena `bad-if` za skutočnú špeciálnu formu `if` problém opraví.

3.3 Odložené vyhodnocovanie

Odložené vyhodnocovanie, po anglicky *delayed evaluation*, znamená, že výpočet argumentu sa nevykoná okamžite, ale odloží sa na neskôr.

Namiesto hodnoty sa funkcii odovzdá výpočet zabalený do nulárnej funkcie. Takejto nulárnej funkcii sa hovorí *thunk*.

```
type Thunk a = () -> a
```

V Haskellí by sme si `thunk` mohli predstaviť ako hodnotu typu:

```
() -> a
```

V Racket štýle:

```
(lambda () expensive-computation)
```

Teda výraz sa nevyhodnotí hneď. Vyhodnotí sa až vtedy, keď zavoláme túto funkciu bez argumentov.

3.4 Thunks v striktných jazykoch

V striktnom jazyku môžeme odložiť vyhodnotenie tým, že výraz zabalíme do funkcie.

Príklad v Racket štýle:

```
(define (mult x y-thunk)
  (cond [(= x 0) 0]
        [(= x 1) (y-thunk)]
        [else (+ (y-thunk)
                  (mult (sub1 x) y-thunk))]))

(define (mul10 x)
  (mult x (lambda () (slow 10))))
```

Tu `y-thunk` nie je priamo hodnota, ale výpočet, ktorý možno spustiť zavolaním:

```
(y-thunk)
```

Výhoda je, že ak `x = 0`, výpočet `slow 10` sa vôbec nevykoná. Nevýhoda je, že ak sa `thunk` použije viackrát, môže sa viackrát vyhodnotiť. Materiál ukazuje, že pri `mul10 5` sa výpočet

3.5 Lenivé vyhodnocovanie

Lenivé vyhodnocovanie, po anglicky *lazy evaluation*, je odložené vyhodnocovanie doplnené o zapamätanie výsledku.

Znamená to:

- výraz sa nevyhodnotí, kým ho naozaj netreba,
- keď sa vyhodnotí, jeho výsledok sa uloží,
- pri ďalšom použití sa už nepočíta znova, ale použije sa uložená hodnota.

Lenivé vyhodnocovanie sa nazýva aj *call-by-need*. Haskell je typickým príkladom jazyka s lenivým vyhodnocovaním.

Rozdiel medzi odloženým a lenivým vyhodnocovaním:

Odložené vyhodnocovanie	Lenivé vyhodnocovanie
Výpočet sa odloží.	Výpočet sa odloží.
Pri každom použití sa môže vykonať znova.	Výsledok sa po prvom výpočte zapamätá.
Call-by-name.	Call-by-need.

3.6 Implementácia lenivosti pomocou thunkov

Lenivú hodnotu možno implementovať ako štruktúru obsahujúcu:

- informáciu, či už bola hodnota vyhodnotená,
- buď thunk, alebo už vypočítanú hodnotu.

V Racket materiáloch sa uvádza implementácia pomocou `delay` a meniteľných párov `mcons`. Makro `delay` zabalí výraz do thunku a uloží ho do štruktúry:

```
(define-syntax delay
  (syntax-rules ()
    [(delay e) (mcons #f (lambda () e))]))
```

Myšlienka je, že `#f` označuje, že hodnota ešte nebola vypočítaná, a druhá zložka obsahuje thunk. Po vynútení výpočtu sa thunk nahradí

3.7 Striktnosť v lenivom jazyku

Hoci Haskell je lenivý, niekedy potrebujeme vyhodnotenie vynútiť. Typickým dôvodom je efektívnosť: nechceme vytvárať veľké reťaze nevyhodnotených výrazov.

Na základné vynútenie vyhodnotenia slúži funkcia:

```
seq :: a -> b -> b
```

Výraz:

```
x 'seq' y
```

najprv vyhodnotí `x` do WHNF a potom vráti `y`. WHNF znamená *weak head normal form*, teda hodnota je vyhodnotená natoľko, aby bol známy jej vonkajší konštruktor.

V materiáloch je `seq` označené ako najzákladnejší spôsob zavedenia

3.8 foldl

Funkcia `foldl` spracúva zoznam zľava doprava.

Jej typ je:

```
foldl :: (b -> a -> b) -> b -> [a] -> b
```

Definícia:

```
foldl _ e [] = e
foldl f e (x:xs) = foldl f (f e x) xs
```

Pre zoznam `[x1,x2,x3,x4]` platí:

```
foldl f e [x1,x2,x3,x4]
= (((e 'f' x1) 'f' x2) 'f' x3) 'f' x4
```

Materiály uvádzajú napríklad:

```
sum      = foldl (+) 0
product = foldl (*) 1
reverse = foldl (flip (:)) []
length  = foldl inc 0
  where inc n x = n + 1
```

3.9 Problém foldl v lenivom jazyku

V Haskellu je `foldl` lenivé v akumulátore. To môže viesť k tomu, že namiesto priebežného výpočtu výsledku sa vytvorí veľký thunk.

Napríklad:

```
foldl (+) 0 [1,2,3,4]
```

sa môže správať približne ako vytváranie výrazu:

```
((((0 + 1) + 2) + 3) + 4)
```

Tento výraz sa nemusí vyhodnocovať priebežne, ale až na konci. Pri veľkých zoznamoch tak vzniká veľa nevyhodnotených výrazov, čo môže viesť k veľkej spotrebe pamäte alebo až k pretečeniu zásobníka/pamäte.

Preto pre striktné akumulčné výpočty, napríklad sčítanie čísel, často nechceme použiť `foldl`, ale `foldl'`.

3.10 foldl'

Funkcia `foldl'` je striktnější verzia `foldl`. Priebežne vynucuje vyhodnotenie akumulátora.

Definícia z materiálov:

```
foldl' :: (a -> b -> a) -> a -> [b] -> a
foldl' _ z []      = z
foldl' f z (x:xs) =
  let z' = f z x
  in z' `seq` foldl' f z' xs
```

Rozdiel je v použití `seq`. Najprv sa vypočíta nový akumulátor `z'`, potom sa vynúti jeho vyhodnotenie a až potom sa pokračuje rekurzívne.

Preto:

```
foldl' (+) 0 [1..100000000]
```

je vhodnejšie ako:

```
foldl (+) 0 [1..100000000]
```

ak chceme striktné spočítať veľký konečný zoznam. Materiál odporúča použiť `foldl'` vtedy, keď je kombinačná funkcia striktná, napríklad `(+)`.

3.11 foldr vs. foldl'

Funkcia `foldr` spracúva zoznam sprava:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr _ e []      = e
foldr f e (x:xs) = x 'f' foldr f e xs
```

Pre zoznam `[x1,x2,x3]`:

```
foldr f e [x1,x2,x3]
= x1 'f' (x2 'f' (x3 'f' e))
```

V lenivom jazyku má `foldr` veľkú výhodu: ak je funkcia `f` lenivá vo svojom druhom argumente, `foldr` môže fungovať aj s nekonečnými zoznamami.

Príklad:

```
foldr (&&) True (False : repeat True)
```

Výsledok je `False`, pretože `()` pri prvom argumente `False` nepotrebuje vyhodnotiť druhý argument.

Pravidlo:

- `foldr` používame, keď kombinačná funkcia môže byť lenivá v druhom argumente alebo keď pracujeme s nekonečnými zoznamami,
- `foldl'` používame pri striktnej akumulácii nad konečnými zoznamami, napríklad pri sčítaní.

3.12 Prečo foldl' nemusí vždy stačiť

`foldl'` vynucuje akumulátor iba do WHNF. To znamená, že ak je akumulátor napríklad dvojica, `seq` vynúti iba vonkajší konštruktor dvojice, ale nie nutne jej zložky.

Nesprávny príklad:

```
f (acc, len) x = (acc + x, len + 1)
```

```
foldl' f (0, 0) [1,2,3]
```

Problém je, že dvojica sa síce vytvorí, ale jej zložky môžu ostať ako nevyhodnotené výrazy.

Lepšia verzia:

```
f' (acc, len) x =
  let acc' = acc + x
      len' = len + 1
  in acc' 'seq' len' 'seq' (acc', len')
```

```
foldl' f' (0, 0) [1,2,3]
```

Tu sa explicitne vynútiť aj obe zložky akumulátora. Tento príklad je priamo uvedený v materiáloch k `seq` a `foldl'`.

3.13 Meniteľné dátové štruktúry

Meniteľná dátová štruktúra, po anglicky *mutable data structure*, je štruktúra, ktorú možno po vytvorení zmeniť na mieste, teda *in-place*.

Príklady:

- pole v C alebo Java,
- objekt s meniteľnými atribútmi,
- meniteľný zoznam,
- meniteľná referencia.

Typická operácia:

```
arr[i] = newValue
```

nemení len lokálnu premennú, ale samotnú dátovú štruktúru. Ak na tú istú štruktúru existuje viac aliasov, zmenuvidia všetky.

3.14 Problém aliasingu

Ak dve premenné ukazujú na tú istú meniteľnú štruktúru, zmena cez jednu premennú sa prejaví aj cez druhú.

Príklad v pseudokóde:

```
xs = [1,2,3]
ys = xs
xs[0] = 99
```

Potom aj `ys` môže vidieť zoznam:

```
[99,2,3]
```

Toto komplikuje uvažovanie o programe, pretože hodnota sa môže zmeniť “potichu” cez iný alias.

3.15 Nemenné dátové štruktúry

Nemenná dátová štruktúra, po anglicky *immutable data structure*, sa po vytvorení už nedá zmeniť. Namiesto zmeny pôvodnej štruktúry sa vytvorí nová štruktúra, ktorá z pôvodnej vychádza.

Materiál uvádza tieto vlastnosti nemenných dátových štruktúr:

- po vytvorení už nemôžu byť modifikované,
- môžeme vytvoriť novú štruktúru vychádzajúcu zo starej,
- starú štruktúru nemožno zmeniť *in-place*,

- nové verzie nemôžu byť prístupné cez staré aliasy.

Príklad:

```
xs = [1,2,3]
ys = update xs 0 99
```

Potom:

```
xs == [1,2,3]
ys == [99,2,3]
```

Pôvodná štruktúra ostáva nezmenená.

3.16 Výhody nemenných dátových štruktúr

Nemenné dátové štruktúry majú viaceré výhody:

- ľahšie sa o nich uvažuje,
- majú iba jeden stav,
- zabráňujú skrytým vedľajším efektom,
- sú bezpečnejšie vo viacvláknovom prostredí,
- možno ich zdieľať medzi viacerými časťami programu,
- stará a nová verzia môžu zdieľať spoločné časti.

Zdieľanie spoločných častí je dôležité pre efektívnosť. Nemenná štruktúra nemusí pri každej zmene kopírovať všetko. Môže vytvoriť novú verziu, ktorá väčšinu uzlov zdieľa so starou verziou.

3.17 Persistentné dátové štruktúry

Nemenné dátové štruktúry bývajú často persistentné. To znamená, že po operácii zmeny sú dostupné obe verzie: stará aj nová.

Príklad:

```
xs = [1,2,3]
ys = cons 0 xs
```

Potom:

```
xs == [1,2,3]
ys == [0,1,2,3]
```

Zoznam `ys` môže zdieľať chvost so zoznamom `xs`. To je efektívne, pretože sa nemusí kopírovať celý zoznam.

3.18 Mutable a immutable v Rackete

Racket má aj funkcionálne, aj imperatívne vlastnosti. Podporuje meniteľné premenné cez `set!`. Materiál uvádza príklad:

```
(define a 10)
(define f (lambda (x) (+ x a)))
(set! a 7)
(define r1 (f 4))
```

Výsledok `r1` je 11, pretože premenná `a` bola zmenená na 7.

Racket zároveň rozlišuje medzi bežnými nemennými párami a meniteľnými párami. Bežné `cons` bunky sú nemenné, zatiaľ čo pre meniteľné páry používa:

```
mcons
mcar
mcdr
set-mcar!
set-mcdr!
```

Príklad:

```
(define mpair (mcons 1 (mcons #t "a")))
(set-mcar! (mcdr mpair) "boo")
```

Po zmene sa vnútorný prvok zmení z `#t` na `"boo"`. Materiál tiež upozorňuje, že funkcie pre `mcons` a bežné `cons` bunky nie sú zameniteľné.

3.19 Mutable štruktúry v Haskellí

Haskell preferuje nemenné dátové štruktúry, ale podporuje aj kontrolovanú mutabilitu, napríklad cez `IORef`, `STRef`, mutable arrays alebo monádu `ST`.

Príklad z materiálov:

```
import Control.Monad.ST (runST)
import Data.STRef (newSTRef, modifySTRef', readSTRef)
import Data.Foldable (for_)

sumST :: (Foldable t, Num a) => t a -> a
sumST xs = runST $ do
  n <- newSTRef 0
  for_ xs $ \x ->
    modifySTRef' n (+x)
  readSTRef n
```

Tu sa interne používa meniteľná referencia, ale navonok je funkcia `sumST` čistá. Monáda `ST` zabezpečí, že mutabilita neunikne mimo lokálneho výpočtu.

3.20 Porovnanie

Pojem	Význam
Striktné vyhodnocovanie	Argumenty sa vyhodnotia pred zavolaním funkcie.
Odložené vyhodnocovanie	Argument sa zabalí do výpočtu a vyhodnotí sa až pri použití.
Thunk	Nulárna funkcia alebo objekt reprezentujúci odložený výpočet.
Lenivé vyhodnocovanie	Odložené vyhodnocovanie so zapamätaním výsledku.
<code>foldl</code>	Ľavý fold, v Haskellí lenivý v akumulátore.
<code>foldl'</code>	Striktný ľavý fold, priebežne vynucuje akumulátor.
<code>seq</code>	Základný spôsob zavedenia striktnosti v Haskellí.
Mutable štruktúra	Dá sa meniť po vytvorení in-place.
Immutable štruktúra	Po vytvorení sa nemení; zmeny vytvárajú nové verzie.

3.21 Krátke zhrnutie

Striktné vyhodnocovanie vyhodnotí argumenty pred zavolaním funkcie. Odložené vyhodnocovanie argument nevyhodnotí hneď, ale zabalí ho do thunku. Lenivé vyhodnocovanie je odložené vyhodnocovanie so zapamätaním výsledku, takže výraz sa vyhodnotí najviac raz. V striktných jazykoch možno odkladanie simulovať thunkmi. V Haskellí je vyhodnocovanie prirodzene lenivé, ale často treba zaviesť striktnosť pomocou `seq` alebo `foldl'`. Meniteľné dátové štruktúry možno meniť in-place, čo môže viesť k problémom s aliasingom a vedľajšími efektmi. Nemenné dátové štruktúry sa po vytvorení nemenia, ľahšie sa zdieľajú a bezpečnejšie sa používajú vo funkcionálnom a paralelnom programovaní.

4 Návrhový vzor monad a jeho využitie: Maybe, List, Reader, Writer, State

4.1 Motivácia

Monáda je návrhový vzor, ktorý umožňuje skladať výpočty, ktoré okrem obvyčajnej hodnoty nesú aj nejaký kontext.

Bežné funkcie vieme skladať pomocou operátora `(.)`:

```
(.) :: (b -> c) -> (a -> b) -> a -> c
```

Ak však funkcie nevracajú obvyčajnú hodnotu, ale hodnotu v kontexte, obvyčajné skladanie už nestačí.

Príklady funkcií s kontextom:

```
a -> Maybe b      -- výpočet môže zlyhať
a -> [b]           -- výpočet môže mať viac výsledkov
a -> r -> b        -- výpočet číta spoločné prostredie
```

```

a -> (w, b)      -- výpočet produkuje log
a -> s -> (a, s)  -- výpočet pracuje so stavom

```

Monáda poskytuje jednotné rozhranie na skladanie takýchto výpočtov.

4.2 Monáda ako typová trieda

V Haskellí je monáda reprezentovaná typovou triedou:

```

class Applicative m => Monad m where
  (>>=)  :: m a -> (a -> m b) -> m b
  (>>)   :: m a -> m b -> m b
  return :: a -> m a

```

Najdôležitejšie operácie sú:

- **return**, ktorá vloží čistou hodnotu do monadického kontextu,
- (**>>=**), nazývaná *bind*, ktorá zoberie hodnotu v kontexte a odovzdá jej vnútornú hodnotu funkcii, ktorá opäť vracia hodnotu v kontexte.

Typ operátora **>>=** je:

```

(>>=) :: m a -> (a -> m b) -> m b

```

Číta sa takto: máme hodnotu typu **m a**; ak sa z nej dá získať hodnota typu **a**, odovzdáme ju funkcii typu **a -> m b** a výsledkom je hodnota typu **m b**.

4.3 Monadické zákony

Aby sa typ správal ako monáda korektne, implementácie **return** a **>>=** majú spĺňať tri monadické zákony.

4.3.1 Ľavá identita

```

return a >>= k = k a

```

Ak vložíme hodnotu **a** do monády a hneď ju odovzdáme funkcii **k**, je to rovnaké, ako keby sme zavolali **k a** priamo.

4.3.2 Pravá identita

```

m >>= return = m

```

Ak z monadickej hodnoty vyberieme hodnotu **a** hneď ju vložíme naspäť pomocou **return**, nič sa nezmení.

4.3.3 Asociativita

```
m >>= (\x -> k x >>= h) = (m >>= k) >>= h
```

Pri skladaní monadických výpočtov nezáleží na uzátvorkovaní.

4.4 Kleisliho skladanie

Monády možno chápať aj ako spôsob skladania funkcií tvaru:

```
a -> m b
```

Takéto funkcie sa nazývajú Kleisliho šípky. Ich skladanie má typ:

```
(<=<) :: Monad m => (b -> m c) -> (a -> m b) -> a -> m c
```

Intuitívne:

```
(f <=< g) x = g x >>= f
```

Teda najprv vykonáme `g x`; ak vznikne výsledok v monadickom kontexte, pokračujeme funkciou `f`.

4.5 do-notácia

Pretože zápis pomocou `>>=` môže byť neprehľadný, Haskell poskytuje `do`-notáciu.

Výraz:

```
do
  x <- mx
  y <- my
  return (x + y)
```

je syntaktický cukor pre:

```
mx >>= \x ->
my >>= \y ->
return (x + y)
```

Všeobecné pravidlá:

```
do { e }           = e
do { e; stmts }    = e >> do { stmts }
do { p <- e; stmts } = e >>= \p -> do { stmts }
```

4.6 Maybe monáda

Typ `Maybe` reprezentuje výpočet, ktorý môže zlyhať.

```
data Maybe a = Nothing | Just a
```

Význam:

- `Just x` znamená úspešný výsledok `x`,
- `Nothing` znamená zlyhanie alebo chýbajúcu hodnotu.

Monadická inštancia:

```
instance Monad Maybe where
  Just x  >>= k = k x
  Nothing >>= _ = Nothing
```

Ak je výsledkom `Just x`, pokračujeme vo výpočte. Ak je výsledkom `Nothing`, ďalšie výpočty sa preskočia a výsledkom ostáva `Nothing`.

4.6.1 Príklad: bezpečné delenie

```
safeDiv :: Double -> Double -> Maybe Double
safeDiv _ 0 = Nothing
safeDiv x y = Just (x / y)
```

Bez monád by sme museli ručne kontrolovať každý krok:

```
calc :: Double -> Double -> Double -> Maybe Double
calc x y z =
  case safeDiv x y of
    Nothing -> Nothing
    Just r1 ->
      case safeDiv r1 z of
        Nothing -> Nothing
        Just r2 -> Just r2
```

Pomocou `do`-notácie:

```
calc :: Double -> Double -> Double -> Maybe Double
calc x y z = do
  r1 <- safeDiv x y
  r2 <- safeDiv r1 z
  return r2
```

Ak niektorý krok vráti `Nothing`, celý výpočet skončí ako `Nothing`.

4.7 List monáda

Zoznamová monáda reprezentuje nedeterminizmus alebo viacero možných výsledkov.

Monadická inštancia pre zoznamy:

```
instance Monad [] where
  xs >>= f = [ y | x <- xs, y <- f x ]
```

To znamená: pre každý prvok x zo zoznamu xs sa vypočíta zoznam výsledkov $f\ x$; všetky tieto zoznamy sa následne spoja.

4.7.1 Príklad

```
pairs :: [(Int, Int)]
pairs = do
  x <- [1,2,3]
  y <- [10,20]
  return (x, y)
```

Výsledok:

```
[(1,10),(1,20),(2,10),(2,20),(3,10),(3,20)]
```

Tento zápis je ekvivalentný zoznamovej komprehézii:

```
pairs = [ (x,y) | x <- [1,2,3], y <- [10,20] ]
```

4.7.2 Závislosť ďalšieho výberu od predchádzajúcej hodnoty

Monády sú silnejšie než aplikatívne funktory, pretože ďalší výpočet môže závisieť od predchádzajúcej hodnoty.

```
products :: [Integer]
products = do
  x <- [1..3]
  y <- [1..x]
  return (x * y)
```

Výsledok:

```
[1,2,4,3,6,9]
```

Tu rozsah $[1..x]$ závisí od hodnoty x , ktorá bola získaná v predchádzajúcom kroku.

4.8 Reader monáda

Reader monáda reprezentuje výpočet, ktorý číta hodnoty zo spoločného prostredia. Typicky sa používa na odovzdávanie konfigurácie, globálneho prostredia alebo závislostí bez explicitného posúvania argumentu cez každú funkciu.

Reader možno chápať ako funkciu:

```
newtype Reader r a = Reader { runReader :: r -> a }
```

Typ `Reader r a` znamená: výpočet, ktorý číta prostredie typu `r` a vráti hodnotu typu `a`.

4.8.1 Monadická intuícia

```
return x
```

vytvorí výpočet, ktorý ignoruje prostredie a vráti `x`.

```
m >>= k
```

spustí výpočet `m` v danom prostredí, výsledok odovzdá funkcii `k` a aj nasledujúci výpočet spustí v tom istom prostredí.

Zjednodušená implementácia:

```
instance Monad (Reader r) where
  return x = Reader $ \_ -> x

  Reader m >>= k = Reader $ \r ->
    let x = m r
    in Reader m' = k x
    in m' r
```

4.8.2 Príklad

```
data Config = Config
  { prefix :: String
  , factor :: Int
  }

type App a = Reader Config a

ask :: Reader r r
ask = Reader id

makeMessage :: String -> App String
makeMessage name = do
  cfg <- ask
  return (prefix cfg ++ " " ++ name)
```

Spustenie:

```
runReader (makeMessage "Janko") (Config "Ahoj" 3)
```

Výsledok:

```
"Ahoj Janko"
```

Reader teda rieši situáciu, keď veľa funkcií potrebuje čítať rovnakú konfiguráciu.

4.9 Writer monáda

Writer monáda reprezentuje výpočet, ktorý okrem výsledku produkuje aj vedľajší výstup, typicky log.

Základná predstava:

```
newtype Writer w a = Writer { runWriter :: (w, a) }
```

Typ `w` musí byť monoid, aby bolo možné spájať logy.

Monoid poskytuje:

```
mempty  :: w  
mappend :: w -> w -> w
```

V modernom Haskellu sa používa aj operátor:

```
(<>) :: w -> w -> w
```

4.9.1 Monadická intuícia

```
return x
```

vráti hodnotu `x` a prázdny log.

```
m >>= k
```

vykoná prvý výpočet, získa jeho log, potom vykoná druhý výpočet a oba logy spojí.

Zjednodušená implementácia:

```
instance Monoid w => Monad (Writer w) where  
  return x = Writer (mempty, x)  
  
  Writer (w1, x) >>= k =  
    let Writer (w2, y) = k x  
    in Writer (w1 <> w2, y)
```

4.9.2 Príklad

```
tell :: w -> Writer w ()
tell w = Writer (w, ())

half :: Int -> Writer [String] Int
half x = do
    tell ["Halving " ++ show x]
    return (x `div` 2)

double :: Int -> Writer [String] Int
double x = do
    tell ["Doubling " ++ show x]
    return (x * 2)

doubleHalf :: Int -> Writer [String] Int
doubleHalf x = do
    y <- half x
    double y
```

Spustenie:

```
runWriter (doubleHalf 9)
```

Výsledok:

```
(["Halving 9", "Doubling 4"], 8)
```

Writer sa hodí napríklad na logovanie, zbieranie štatistík alebo zbieranie varovaní počas výpočtu.

4.10 State monáda

State monáda reprezentuje výpočet, ktorý pracuje so stavom. Čistým spôsobom modeluje imperatívne programovanie so stavom.

Základná predstava:

```
newtype State s a = State { runState :: s -> (a, s) }
```

Typ `State s a` znamená: výpočet, ktorý dostane stav typu `s` a vráti hodnotu typu `a` spolu s novým stavom typu `s`.

4.10.1 Monadická intuícia

```
return x
```

vráti hodnotu `x` bez zmeny stavu.

```
m >>= k
```

spustí prvý stavový výpočet, získa jeho výsledok a nový stav, potom spustí ďalší výpočet s týmto novým stavom.

Zjednodušená implementácia:

```
instance Monad (State s) where
    return x = State $ \s -> (x, s)

    State m >>= k = State $ \s ->
        let (x, s') = m s
        in State m' = k x
        in m' s'
```

4.10.2 Základné operácie

```
get :: State s s
get = State $ \s -> (s, s)

put :: s -> State s ()
put s = State $ \_ -> ((), s)

modify :: (s -> s) -> State s ()
modify f = State $ \s -> ((), f s)
```

4.10.3 Príklad: čítač

```
tick :: State Int Int
tick = do
    n <- get
    put (n + 1)
    return n
```

Spustenie:

```
runState tick 10
```

Výsledok:

```
(10, 11)
```

Výpočet vrátil pôvodný stav ako hodnotu a zároveň zvýšil stav o jedna.

4.10.4 Príklad: očíslovanie prvkov zoznamu

```
label :: [a] -> State Int [(Int, a)]
label [] = return []
label (x:xs) = do
    n <- get
    put (n + 1)
    ys <- label xs
    return ((n, x) : ys)
```

Spustenie:

```
runState (label ["a","b","c"]) 0
```

Výsledok:

```
((0,"a"),(1,"b"),(2,"c")), 3)
```

State monáda sa hodí tam, kde by sme v imperatívnom jazyku používali premennú meniacu sa počas výpočtu.

4.11 Porovnanie monád

Monáda	Kontext	Typické použitie
Maybe	možnosť zlyhania	bezpečné delenie, hľadanie, parsovanie
[]	viac výsledkov	nedeterminizmus, generovanie kombinácií
Reader r	spoločné prostredie	konfigurácia, dependency injection
Writer w	dodatočný výstup	logovanie, zbieranie správ, štatistiky
State s	meniaci sa stav	čítače, generátory, simulácia imperatívneho stavu

4.12 Monády verzus funktory a aplikatívne funktory

Funktor umožňuje aplikovať čistú funkciu na hodnotu v kontexte:

```
fmap :: (a -> b) -> f a -> f b
```

Aplikatívny funktor umožňuje aplikovať funkciu v kontexte na hodnotu v kontexte:

```
(<*>) :: f (a -> b) -> f a -> f b
```

Monáda umožňuje, aby ďalší výpočet závisel od predchádzajúcej obyčajnej hodnoty:

```
(>=>) :: m a -> (a -> m b) -> m b
```

Práve toto je hlavná sila monád. Funkcia `a -> m b` môže vytvoriť nový kontext na základe konkrétnej hodnoty `a`.

4.13 Prečo sú monády návrhový vzor

Monáda nie je iba konkrétny typ, ale všeobecný spôsob organizácie výpočtov. Pre rôzne monády znamená “kontext” niečo iné:

- pri `Maybe` je kontext úspech alebo zlyhanie,
- pri `zozname` je kontext množina možností,

- pri **Reader** je kontext spoločné prostredie,
- pri **Writer** je kontext dodatočný výstup,
- pri **State** je kontext stav, ktorý sa prenáša výpočtom.

Operácie **return** a **>>=** však pre všetky tieto prípady poskytujú rovnaký štýl skladania.

4.14 Krátke zhrnutie

Monáda je návrhový vzor na skladanie výpočtov v kontexte. Operácia **return** vloží čistou hodnotu do kontextu a operácia **>>=** umožní pokračovať vo výpočte na základe hodnoty získanej z predchádzajúceho kontextu. **Maybe** modeluje zlyhanie, zoznam modeluje viacero výsledkov, **Reader** čítanie prostredia, **Writer** produkciu logu a **State** prácu so stavom. **do**-notácia je syntaktický cukor nad **>>=** a umožňuje zapisovať monadické výpočty prehľadne, podobne ako sekvenciu príkazov.

5 Prvorádové funkcie, funkcie vyššieho rádu, uzávery a typová inferencia

5.1 Prvorádové funkcie

Funkcie sú v jazyku *first-class*, po slovensky prvorádové alebo prvotriedne, ak sa s nimi dá zaobchádzať rovnako ako s ostatnými hodnotami.

To znamená, že funkciu možno:

- uložiť do premennej,
- odovzdať ako argument inej funkcii,
- vrátiť ako výsledok funkcie,
- uložiť do dátovej štruktúry,
- anonymne vytvoriť pomocou lambda výrazu.

V Haskellí sú funkcie prvorádové hodnoty.

```
inc :: Int -> Int
inc x = x + 1
```

Funkciu **inc** môžeme odovzdať ako argument:

```
map inc [1,2,3]
-- [2,3,4]
```

Funkciu môžeme vytvoriť anonymne:

```
map (\x -> x + 1) [1,2,3]
-- [2,3,4]
```

Funkciu môžeme aj vrátiť ako výsledok:

```
add :: Int -> (Int -> Int)
add x = \y -> x + y
```

Potom:

```
add 3 4 == 7
```

5.2 Aplikácia funkcie v Haskell

Aplikácia funkcie sa v Haskell zapisuje jednoducho medzerou:

```
f x
```

Aplikácia je zľava asociatívna:

```
f x y z == ((f x) y) z
```

Preto zápis:

```
add 3 4
```

znamená:

```
((add 3) 4)
```

Toto súvisí s curryingom: viacargumentové funkcie možno chápať ako funkcie, ktoré postupne vracajú ďalšie funkcie.

5.3 Currying a čiastočná aplikácia

V Haskell má funkcia:

```
plus :: Int -> Int -> Int
plus x y = x + y
```

typ, ktorý sa asociuje doprava:

```
Int -> Int -> Int == Int -> (Int -> Int)
```

To znamená, že `plus` vezme jeden argument typu `Int` a vráti funkciu typu `Int -> Int`.

Príklad čiastočnej aplikácie:

```
plus3 :: Int -> Int
plus3 = plus 3
```

Potom:

```
plus3 10 == 13
```

Čiastočná aplikácia je dôležitá pri funkciách vyššieho rádu. Napríklad:

```
map (+1) [1,2,3]
```

Tu (+1) je funkcia, ktorá ku svojmu argumentu pripočíta jednotku.

5.4 Funkcie vyššieho rádu

Funkcia vyššieho rádu je funkcia, ktorá aspoň jednu funkciu berie ako argument alebo funkciu vracia ako výsledok.

Príklady:

```
map    :: (a -> b) -> [a] -> [b]
foldr  :: (a -> b -> b) -> b -> [a] -> b
foldl  :: (b -> a -> b) -> b -> [a] -> b
filter :: (a -> Bool) -> [a] -> [a]
(.)    :: (b -> c) -> (a -> b) -> a -> c
```

Funkcie vyššieho rádu umožňujú abstraktne vyjadriť opakujúce sa vzory výpočtov. Namiesto toho, aby sme stále písali rekurziu nad zoznamom, použijeme vhodnú všeobecnú funkciu.

5.5 Funkcia map

Funkcia `map` aplikuje danú funkciu na každý prvok zoznamu.

```
map :: (a -> b) -> [a] -> [b]
map _ []      = []
map f (x:xs) = f x : map f xs
```

Význam typu:

- `f :: a -> b` je funkcia, ktorá mení prvok typu `a` na hodnotu typu `b`,
- `[a]` je vstupný zoznam,
- `[b]` je výsledný zoznam.

Príklady:

```
map (*2) [1,2,3]
-- [2,4,6]
```

```
map show [1,2,3]
-- ["1","2","3"]
```

```
map length ["aa", "b", "abcd"]
-- [2,1,4]
```

Dôležité je, že `map` nemení dĺžku zoznamu. Mení iba jednotlivé hodnoty.

5.6 Funkcia foldr

Funkcia `foldr` je pravý fold. Spracúva zoznam tak, že nahradí konštruktor zoznamu `(:)` zadanou kombinačnou funkciou a prázdny zoznam zadanou počiatočnou hodnotou.

Typ:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
```

Definícia:

```
foldr _ e []      = e
foldr f e (x:xs) = x 'f' foldr f e xs
```

Pre zoznam `[x1,x2,x3,x4]` platí:

```
foldr f e [x1,x2,x3,x4]
= x1 'f' (x2 'f' (x3 'f' (x4 'f' e)))
```

Príklady:

```
sum      = foldr (+) 0
product = foldr (*) 1
concat   = foldr (++) []
and       = foldr (&&) True
or        = foldr (||) False
```

Pomocou `foldr` možno definovať aj `map`:

```
map f = foldr ((:) . f) []
```

Ekvivalentne:

```
map f = foldr (\x xs -> f x : xs) []
```

5.7 Intuícia foldr

Zoznam:

```
[1,2,3]
```

je syntaktický cukor pre:

```
1 : 2 : 3 : []
```

Ak použijeme:

```
foldr (+) 0 [1,2,3]
```

dostaneme:

`1 + (2 + (3 + 0))`

Ak použijeme:

`foldr (:) [] [1,2,3]`

dostaneme pôvodný zoznam:

`1 : (2 : (3 : []))`

5.8 Funkcia foldl

Funkcia `foldl` je ľavý fold. Spracúva zoznam zľava doprava a nesie si priebežný akumulátor.

Typ:

`foldl :: (b -> a -> b) -> b -> [a] -> b`

Definícia:

```
foldl _ e []      = e
foldl f e (x:xs) = foldl f (f e x) xs
```

Pre zoznam `[x1,x2,x3,x4]` platí:

```
foldl f e [x1,x2,x3,x4]
= (((e 'f' x1) 'f' x2) 'f' x3) 'f' x4
```

Príklady:

```
sum      = foldl (+) 0
product  = foldl (*) 1
length   = foldl (\n _ -> n + 1) 0
reverse  = foldl (flip (:)) []
```

5.9 Rozdiel medzi foldr a foldl

Hlavný rozdiel je v asociovaní výpočtu.

```
foldr f e [1,2,3] = 1 'f' (2 'f' (3 'f' e))
foldl f e [1,2,3] = ((e 'f' 1) 'f' 2) 'f' 3
```

Pre asociatívne operácie s neutrálnym prvkom, napríklad sčítanie, dávajú rovnaký výsledok:

```
foldr (+) 0 [1,2,3] == 6
foldl (+) 0 [1,2,3] == 6
```

Pre neasociatívne operácie sa môžu líšiť:

```
foldr (-) 0 [1,2,3] == 1 - (2 - (3 - 0)) == 2
foldl (-) 0 [1,2,3] == ((0 - 1) - 2) - 3 == -6
```

5.10 Kedy použiť foldr a kedy foldl

`foldr` je prirodzený pri rekurzii nad zoznamami, ktorá kopíruje štruktúru zoznamu alebo môže skončiť bez vyhodnotenia celého zvyšku. V lenivom jazyku vie `foldr` pracovať aj s nekonečnými zoznamami, ak kombinačná funkcia nepotrebuje vždy svoj druhý argument.

Príklad:

```
foldr (&&) True (False : repeat True)
-- False
```

Funkcia `()` pri prvom argumente `False` nepotrebuje vyhodnotiť druhý argument.

`foldl` používa akumulátor a je prirodzený pre výpočty typu “prechádzam zoznam zľava doprava a aktualizujem priebežný stav”. V Haskellu sa však pri striktnej akumulácii často používa radšej `foldl'`, pretože obyčajné `foldl` je lenivé v akumulátore.

5.11 Uzávery

Uzáver, po anglicky *closure*, je funkcia spolu s prostredím, v ktorom bola vytvorená. Toto prostredie obsahuje hodnoty voľných premenných, ktoré funkcia používa.

Príklad v Haskellu:

```
makeAdder :: Int -> (Int -> Int)
makeAdder x = \y -> x + y
```

Ak zavoláme:

```
add5 = makeAdder 5
```

vznikne funkcia:

```
\y -> 5 + y
```

Táto funkcia si pamätá hodnotu `x = 5`, hoci volanie `makeAdder 5` už skončilo. Práve uloženie hodnoty `x` v prostredí funkcie je uzáver.

5.12 Voľné a viazané premenné

Vo výraze:

```
\y -> x + y
```

je premenná `y` viazaná lambda abstrakciou, ale `x` je voľná premenná. Aby funkcia mohla byť použitá aj neskôr, musí si voľnú premennú `x` pamätať v uzávere.

V uzávere teda nie je uložený iba kód funkcie, ale aj potrebné prostredie.

5.13 Lexikálny a dynamický scoping

Pri lexikálnom scopingu sa význam premenných určuje podľa miesta, kde je funkcia definovaná. Funkcia používa premenné, ktoré boli viditeľné v čase jej definície.

Pri dynamickom scopingu sa význam premenných určuje podľa miesta, kde je funkcia volaná. Funkcia by teda mohla použiť premenné z aktuálneho volacieho prostredia.

Haskell aj JavaScript používajú lexikálny scoping. Uzávery sú prirodzene spojené s lexikálnym scopingom: funkcia si zapamätá lexikálne dostupné premenné z času definície.

5.14 Príklad uzáveru v JavaScripte

Typický príklad uzáveru:

```
function Container(priv_value) {
    var priv_counter = 2;

    function priv_dec() {
        if (priv_counter > 0) {
            priv_counter -= 1;
            return true;
        } else {
            return false;
        }
    }

    this.getValue = function () {
        return priv_dec() ? priv_value : null;
    };
}
```

```
x = new Container(42);
```

Volania:

```
x.getValue()
x.getValue()
x.getValue()
```

vrátia postupne:

```
42
42
null
```

5.15 Typová inferencia

Typová inferencia znamená automatické odvodenie typu výrazu alebo funkcie kompilátorom. Programátor nemusí vždy písať typové anotácie, pretože kompilátor vie typ často vypočítať zo samotnej definície.

Príklad:

```
inc x = x + 1
```

Haskell vie odvodiť typ:

```
inc :: Num a => a -> a
```

Dôvod je, že operátor (+) pracuje nad číselnými typmi. Preto musí byť x číselného typu a výsledok má rovnaký typ.

Ďalší príklad:

```
len xs = length xs
```

Typ:

```
len :: [a] -> Int
```

Funkcia len nezávisí od konkrétneho typu prvkov zoznamu, preto je polymorfná.

5.16 Typová inferencia pri map

Uvažujme:

```
map f []      = []
map f (x:xs) = f x : map f xs
```

Z definície vieme odvodiť:

- x má nejaký typ a,
- xs má typ [a],
- f x má nejaký typ b,
- teda $f :: a \rightarrow b$,
- výsledkom je zoznam hodnôt typu b.

Preto:

```
map :: (a -> b) -> [a] -> [b]
```

5.17 Typová inferencia pri foldr

Definícia:

```
foldr _ e []      = e
foldr f e (x:xs) = f x (foldr f e xs)
```

Odvodenie typu:

- vstupný zoznam má typ `[a]`,
- prvok `x` má typ `a`,
- počiatočná hodnota `e` má typ `b`,
- výsledok `foldr f e xs` má tiež typ `b`,
- funkcia `f` berie prvok typu `a` a akumulovaný výsledok typu `b` a vracia `b`.

Preto:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
```

5.18 Typová inferencia pri foldl

Definícia:

```
foldl _ e []      = e
foldl f e (x:xs) = foldl f (f e x) xs
```

Odvodenie typu:

- vstupný zoznam má typ `[a]`,
- prvok `x` má typ `a`,
- akumulátor `e` má typ `b`,
- funkcia `f` berie akumulátor typu `b` a prvok typu `a`,
- výsledkom funkcie `f` je nový akumulátor typu `b`.

Preto:

```
foldl :: (b -> a -> b) -> b -> [a] -> b
```

5.19 Parametrický polymorfizmus a typová inferencia

Typová inferencia často vedie k polymorfným typom. Napríklad:

```
identity x = x
```

má typ:

```
identity :: a -> a
```

Funkcia funguje pre ľubovoľný typ **a**. Takýto polymorfizmus sa nazýva parametrický polymorfizmus.

Ďalší príklad:

```
const x y = x
```

má typ:

```
const :: a -> b -> a
```

Typové premenné **a** a **b** hovoria, že funkcia je všeobecná a nezávisí od konkrétnych typov.

5.20 Typové triedy pri inferencii

Ak funkcia používa operácie obmedzené typovou triedou, odvodený typ obsahuje kontext.

Príklad:

```
eqSelf x = x == x
```

Typ:

```
eqSelf :: Eq a => a -> Bool
```

Obmedzenie **Eq a** vzniklo preto, že operátor (**==**) možno použiť iba pre typy, ktoré sú inštanciou typovej triedy **Eq**.

Príklad:

```
bigger x y = x > y
```

Typ:

```
bigger :: Ord a => a -> a -> Bool
```

Obmedzenie **Ord a** vzniklo preto, že (**>**) vyžaduje usporiadateľný typ.

5.21 Prečo sú typové anotácie užitočné

Aj keď Haskell typy často odvodí automaticky, typové anotácie sú užitočné:

- zvyšujú čitateľnosť programu,
- slúžia ako dokumentácia,
- pomáhajú odhaliť chyby,
- umožňujú zúžiť príliš všeobecný typ,
- pri niektorých pokročilých typoch sú nevyhnutné.

Napríklad:

```
factorial :: Integer -> Integer
factorial n = product [1..n]
```

Bez anotácie by Haskell mohol odvodiť všeobecnejší typ. Anotácia hovorí, že chceme konkrétne pracovať s typom `Integer`.

5.22 Obmedzenia typovej inferencie

Haskell používa Hindleyho-Milnerov typový systém, ktorý vie dobre odvodiť bežné polymorfné typy ranku 1. Pre typy vyššieho ranku už všeobecná typová inferencia nefunguje automaticky a treba používať explicitné typové anotácie.

Príklad bežného typu ranku 1:

```
id :: forall a. a -> a
```

Pri typoch ranku 2 alebo vyšších sa `forall` môže objaviť ako súčasť typu argumentu funkcie, napríklad:

```
applyPoly :: (forall a. a -> a) -> (Bool, Char)
```

Takéto typy treba v GHC anotovať explicitne a zapnúť rozšírenie:

```
{-# LANGUAGE RankNTypes #-}
```

5.23 Prepojenie pojmov

Prvorádové funkcie umožňujú, aby funkcie boli obyčajné hodnoty. Vďaka tomu môžu existovať funkcie vyššieho rádu, ako `map`, `foldr` a `foldl`. Lambda výrazy a čiastočná aplikácia často vytvárajú uzávery, pretože výsledná funkcia si pamätá hodnoty z prostredia, v ktorom vznikla. Typová inferencia umožňuje tieto funkcie používať bez toho, aby bolo nutné pri každej definícii ručne písať typ.

5.24 Porovnanie map, foldr a foldl

Funkcia	Typ	Význam
map	$(a \rightarrow b) \rightarrow [a] \rightarrow [b]$	Aplikuje funkciu na každý prvok zoznamu.
foldr	$(a \rightarrow b \rightarrow b) \rightarrow b \rightarrow [a] \rightarrow b$	Pravý fold, prirodzene zodpovedá rekurzii nad zoznamom.
foldl	$(b \rightarrow a \rightarrow b) \rightarrow b \rightarrow [a] \rightarrow b$	Ľavý fold s akumulátorom.

5.25 Krátke zhrnutie

Prvorádové funkcie sú funkcie, s ktorými sa dá pracovať ako s bežnými hodnotami. Funkcie vyššieho rádu berú funkcie ako argumenty alebo ich vracajú ako výsledky. `map` aplikuje funkciu na každý prvok zoznamu, `foldr` skladá zoznam sprava a `foldl` skladá zoznam zľava pomocou akumulátora. Uzáver je funkcia spolu s prostredím, ktoré obsahuje hodnoty jej voľných premenných. Typová inferencia je automatické ododenie typov výrazov a funkcií; v Haskellí často vedie k všeobecným polymorfným typom, prípadne k typom s obmedzeniami cez typové triedy.

6 Funkcie viacerých argumentov, n-tice, curry-ing, čiastočná aplikácia a bezbodový štýl

6.1 Funkcie viacerých argumentov

V mnohých jazykoch poznáme funkcie viacerých argumentov, napríklad:

```
plus(x, y) = x + y
```

Takáto funkcia akoby naraz dostala dva argumenty. V Haskellí je však situácia trochu iná: všetky funkcie sú v jadre unárne, teda berú práve jeden argument. Funkcia viacerých argumentov sa chápe ako funkcia, ktorá vezme jeden argument a vráti ďalšiu funkciu.

Napríklad funkciu sčítania môžeme zapísať takto:

```
plus :: Int -> Int -> Int
plus x y = x + y
```

Typ:

```
Int -> Int -> Int
```

sa asociuje doprava, teda znamená:

```
Int -> (Int -> Int)
```

Funkcia `plus` teda vezme prvé číslo a vráti funkciu, ktorá čaká na druhé číslo.

6.2 Aplikácia funkcie

Aplikácia funkcie sa v Haskellu zapisuje medzerou:

```
plus 2 3
```

Aplikácia je asociatívna zľava:

```
plus 2 3 == (plus 2) 3
```

Najprv sa teda aplikuje `plus` na argument 2. Výsledkom je funkcia:

```
\y -> 2 + y
```

Tá sa potom aplikuje na argument 3 a výsledkom je 5.

6.3 N-tice

N-tica, po anglicky *tuple*, je dátová štruktúra pevnej dĺžky, ktorá môže obsahovať hodnoty rôznych typov.

Príklady:

```
(1, "ahoj")      :: (Int, String)
(True, 3, 'x')    :: (Bool, Int, Char)
("Janko", 4, True) :: (String, Int, Bool)
```

Na rozdiel od zoznamov, ktoré sú homogénne, n-tice môžu obsahovať hodnoty rôznych typov. Zoznam:

```
[1,2,3]
```

má typ:

```
[Int]
```

ale n-tica:

```
(1, "x")
```

má typ:

```
(Int, String)
```

Pre dvojice existujú preddefinované funkcie:

```
fst :: (a, b) -> a
snd :: (a, b) -> b
```

Príklady:

```
fst (1, "x") == 1
snd (1, "x") == "x"
```

N-tice sa používajú vtedy, keď chceme zoskupiť pevný počet hodnôt, často ako jednoduchý návrat viacerých výsledkov z funkcie.

6.4 Funkcia berúca n-ticu

Funkciu viacerých argumentov možno reprezentovať aj ako funkciu jedného argumentu, kde tým argumentom je n-tica.

Príklad:

```
plusTuple :: (Int, Int) -> Int
plusTuple (x, y) = x + y
```

Volanie:

```
plusTuple (2, 3)
```

vráti:

```
5
```

Tu funkcia `plusTuple` naozaj berie iba jeden argument: dvojicu `(Int, Int)`.

Porovnajme:

```
plus      :: Int -> Int -> Int
plusTuple :: (Int, Int) -> Int
```

Prvá funkcia je curried, druhá funkcia berie dvojicu.

6.5 Currying

Currying je transformácia funkcie viacerých argumentov na postupnosť unárnych funkcií.

Napríklad funkciu:

```
plus x y = x + y
```

môžeme chápať ako:

```
plus x = \y -> x + y
```

alebo ešte explicitnejšie:

```
plus = \x -> \y -> x + y
```

Typ:

```
plus :: Int -> Int -> Int
```

znamená:

```
plus :: Int -> (Int -> Int)
```

Teda `plus` po prijatí prvého argumentu vráti funkciu čakajúcu na druhý argument.

6.6 Currying a uncurrying

Prevod medzi curried funkciou a funkciou nad dvojicou zabezpečujú funkcie:

```
curry    :: ((a, b) -> c) -> a -> b -> c
uncurry  :: (a -> b -> c) -> (a, b) -> c
```

Funkcia `curry` zmení funkciu nad dvojicou na curried funkciu:

```
plusTuple :: (Int, Int) -> Int
plusTuple (x, y) = x + y
```

```
plus :: Int -> Int -> Int
plus = curry plusTuple
```

Funkcia `uncurry` zmení curried funkciu na funkciu nad dvojicou:

```
plusTuple2 :: (Int, Int) -> Int
plusTuple2 = uncurry plus
```

Príklady:

```
curry plusTuple 2 3    == 5
uncurry plus (2, 3)    == 5
```

6.7 Čiastočné aplikovanie funkcie

Čiastočná aplikácia znamená, že funkcii nedodáme všetky argumenty naraz. Výsledkom je nová funkcia, ktorá čaká na zvyšné argumenty.

Príklad:

```
plus :: Int -> Int -> Int
plus x y = x + y
```

```
plus10 :: Int -> Int
plus10 = plus 10
```

Funkcia `plus10` vznikla čiastočnou aplikáciou funkcie `plus` na prvý argument 10. Platí:

```
plus10 5 == 15
```

Iné príklady:

```
map (+1) [1,2,3]
-- [2,3,4]
```

```
filter (>0) [-2,0,3,5]
-- [3,5]
```

```
take 3 [1,2,3,4,5]
-- [1,2,3]
```

V zápise:

```
map (+1)
```

sme funkcii `map` dodali iba prvý argument, teda funkciu `(+1)`. Výsledkom je nová funkcia typu:

```
Num a => [a] -> [a]
```

ktorá čaká na zoznam.

6.8 Rozdiel medzi curryingom a čiastočnou aplikáciou

Currying a čiastočná aplikácia spolu úzko súvisia, ale nie sú to tie isté pojmy.

- *Currying* je spôsob reprezentácie funkcie viacerých argumentov ako postupnosti unárnych funkcií.
- *Čiastočná aplikácia* je použitie funkcie na menej argumentov, než koľko potrebuje na úplný výsledok.

Currying umožňuje čiastočnú aplikáciu veľmi prirodzene.

Príklad:

```
plus :: Int -> Int -> Int  
plus x y = x + y
```

Vďaka curryingovému typu:

```
Int -> (Int -> Int)
```

môžeme napísať:

```
plus 10 :: Int -> Int
```

Toto je čiastočná aplikácia.

6.9 Sekcie operátorov

Operátory v Haskellí možno čiastočne aplikovať pomocou sekcií.

Ľavá sekcia:

```
(10+) :: Num a => a -> a  
(10+) x = 10 + x
```

Pravá sekcia:

```
(+10) :: Num a => a -> a  
(+10) x = x + 10
```

Ďalšie príklady:

```
(> 0) 5      == True
(< 10) 7     == True
(/ 2) 10     == 5
(2 /) 10     == 0.2
```

Sekcie sú veľmi užitočné pri funkciách vyššieho rádu:

```
filter (>0) xs
map (*2) xs
all (/=0) xs
```

6.10 Bezbodový štýl

Bezbodový štýl, po anglicky *point-free style*, je štýl zápisu funkcie, v ktorom sa v tele funkcie explicitne nespomínajú jej argumenty. Namiesto toho sa funkcia definuje skladaním iných funkcií.

Bežný zápis:

```
sumList xs = foldr (+) 0 xs
```

Bezbodový zápis:

```
sumList = foldr (+) 0
```

Argument `xs` zmizol z oboch strán rovnice. Tento krok je možný preto, že platí princíp:

```
f x = g x
```

možno prepísať na:

```
f = g
```

ak `x` nie je potrebné explicitne spomínať. Ide o podobnú myšlienku ako η -redukcia v lambda kalkule.

6.11 Skladanie funkcií

Bezbodový štýl často používa operátor skladania funkcií:

```
(.) :: (b -> c) -> (a -> b) -> a -> c
(f . g) x = f (g x)
```

Príklad:

```
f x = not (null x)
```

možno prepísať ako:

```
f = not . null
```

Ďalší príklad:

```
h x = length (filter even x)
```

bezzbodovo:

```
h = length . filter even
```

Tu `filter even` vzniklo čiastočnou aplikáciou funkcie `filter`.

6.12 Ručný prepis do bezbodového štýlu

Uvažujme funkciu:

```
any' p list = or (map p list)
```

Najprv využijeme currying:

```
any' p list = or ((map p) list)
```

Potom skladanie funkcií:

```
any' p list = (or . map p) list
```

Odstránime argument `list`:

```
any' p = or . map p
```

Teraz môžeme odstrániť aj argument `p`. Použijeme fakt, že:

```
or . map p == ((.) or) (map p)
```

a teda:

```
any' p = ((.) or) (map p)
```

Po ďalšom skladaní:

```
any' = ((.) or) . map
```

Ekvivalentne sa dá zapísať:

```
any' = (or .) . map
```

Tento postup je priamo typickým príkladom bezbodového odvodzovania.

6.13 Kedy je bezbodový štýl vhodný

Bezbodový štýl je vhodný, keď skracuje zápis a zvyšuje čitateľnosť.

Dobré príklady:

```
sumList = foldr (+) 0

nonEmpty = not . null

countEven = length . filter even
```

Výhody:

- menej syntaktického šumu,
- menej zbytočných mien premenných,
- zvýraznenie skladania funkcií,
- kratší a často elegantnejší zápis.

6.14 Kedy je bezbodový štýl nevhodný

Bezbodový štýl netreba preháňať. Niekedy sa zápis stane neprehľadným.

Príklad zložitého point-free zápisu:

```
g f x y = (f x, f y)

g = flip =<< (((.) . (,)) .)
```

Hoci ide o platný bezbodový zápis, prvá verzia je pre človeka oveľa čitateľnejšia. Preto je dôležité používať point-free štýl rozumne. Aj materiál upozorňuje, že “point-less style” sa nemá preháňať.

6.15 Porovnanie: curried funkcia vs. funkcia nad n-ticou

Vlastnosť	Curried funkcia	Funkcia nad n-ticou
Typ	<code>a -> b -> c</code>	<code>(a,b) -> c</code>
Počet argumentov navonok	postupne po jednom	jedna dvojica
Čiastočná aplikácia	prirodzená	nie priamo
Volanie	<code>f x y</code>	<code>f (x,y)</code>
Príklad	<code>plus 2 3</code>	<code>plusTuple (2,3)</code>

6.16 Príklady na porovnanie

6.16.1 Curried verzia

```
between :: Int -> Int -> Int -> Bool
between low high x = low <= x && x <= high
```


Použitie:

```
between 1 10 5
-- True
```

Čiastočná aplikácia:

```
small :: Int -> Bool
small = between 1 10

filter small [0,1,5,11]
-- [1,5]
```

6.16.2 Verzia s n-ticou

```
betweenTuple :: (Int, Int, Int) -> Bool
betweenTuple (low, high, x) = low <= x && x <= high
```

Použitie:

```
betweenTuple (1, 10, 5)
-- True
```

Čiastočná aplikácia tu nie je taká prirodzená, pretože funkcia očakáva naraz celú trojicu.

6.17 Krátke zhrnutie

V Haskellu sú všetky funkcie v jadre unárne. Funkcia viacerých argumentov sa zvyčajne chápe ako curried funkcia, teda funkcia, ktorá po jednom argumente vracia ďalšiu funkciu. Alternatívne môže funkcia brať jediný argument, ktorým je n-tica. Currying je transformácia n-árnej funkcie na postupnosť unárnych funkcií, zatiaľ čo čiastočná aplikácia je použitie funkcie na menej argumentov, než koľko potrebuje na úplný výsledok. Bezbodový štýl je zápis funkcie bez explicitného uvádzania argumentov, často pomocou skladania funkcií a čiastočnej aplikácie. Je užitočný, ak zvyšuje čitateľnosť, ale pri príliš komplikovaných výrazoch môže byť naopak neprehľadný.

7 Staticky a dynamicky typované jazyky, jazyky bez typovej kontroly, sémantika príkazu if a zoznamov, duck typing

7.1 Typový systém a typová kontrola

Typový systém je súbor pravidiel, ktoré určujú, aké hodnoty možno používať v akých operáciách. Typová kontrola overuje, či program tieto pravidlá dodržiava.

Príklady typových chýb:

```
1 + "ahoj"  
True * 5  
head 10
```

V rôznych jazykoch sa tieto chyby môžu odhaliť v rôznom čase:

- pred spustením programu, teda staticky,
- počas behu programu, teda dynamicky,
- alebo sa nemusia kontrolovať vôbec a správanie môže byť nedefinované alebo závislé od reprezentácie dát.

7.2 Staticky typované jazyky

Staticky typovaný jazyk kontroluje typy pred spustením programu, typicky počas kompilácie. Ak program nevyhovuje typovým pravidlám, kompilátor ho odmietne.

Príklady staticky typovaných jazykov:

- Haskell,
- Java,
- C#,
- Scala,
- Rust,
- C a C++.

Príklad v Haskell:

```
x = 1 + "ahoj"
```

Takýto program sa neskompiluje, pretože operátor (+) očakáva číselné hodnoty, ale "ahoj" je reťazec.

7.2.1 Výhody statického typovania

- veľa chýb sa odhalí ešte pred spustením programu,
- typy slúžia ako dokumentácia,
- kompilátor môže lepšie optimalizovať,
- vývojové prostredie vie lepšie dopĺňať a refaktorovať kód,
- program má presnejšie špecifikované rozhrania.

7.2.2 Nevýhody statického typovania

- niektoré flexibilné programy sa zapisujú ťažšie,
- typový systém môže vyžadovať dodatočné anotácie,
- niektoré správne programy môže typový systém odmietnuť, pretože ich nevie dostatočne presne vyjadriť.

7.3 Dynamicky typované jazyky

Dynamicky typovaný jazyk kontroluje typy až počas behu programu. Premenné typicky nemajú pevne určený typ; typ majú hodnoty.

Príklady dynamicky typovaných jazykov:

- Racket,
- Python,
- JavaScript,
- Ruby,
- Lua.

V materiáli sa Racket uvádza ako jazyk s dynamickým typovým systémom: počas kompilácie sa nerobí statická typová kontrola, kompilátor akceptuje viac programov, ale niektoré chyby sa objavujú až počas behu.

Príklad v Rackete:

```
(define (sum xs)
  (if (null? xs)
      0
      (+ (car xs) (sum (cdr xs)))))

(define (pokus x)
  (sum (list 1 pokus "tri" 4 x)))
```

Definícia funkcie `pokus` prejde, teda kompilácia je v poriadku. Chyba nastane až pri volaní, keď sa funkcia `sum` pokúsi sčítať číslo s hodnotou, ktorá nie je číslo. V materiáli je výsledkom chyby +: `contract violation`.

7.3.1 Výhody dynamického typovania

- jazyk je často flexibilnejší,
- rýchle prototypovanie,
- ľahšie sa zapisujú heterogénne dátové štruktúry,
- programátor nemusí explicitne písať typy.

7.3.2 Nevýhody dynamického typovania

- niektoré chyby sa prejavajú až počas behu,
- treba viac testovania,
- typové chyby sa môžu objaviť až v zriedkavo používanej vetve programu,
- rozhrania funkcií nemusia byť tak jasné ako pri statických typoch.

7.4 Jazyky bez typovej kontroly

Jazyk bez typovej kontroly alebo s veľmi slabou typovou kontrolou neoveruje dôsledne, či sa hodnota používa spôsobom zodpovedajúcim jej typu.

V extrémnom prípade operácia interpretuje bity hodnoty podľa očakávaného kontextu, aj keď pôvodná hodnota reprezentovala niečo iné. To môže viesť k nedefinovanému správaniu, poškodeniu pamäte alebo bezpečnostným chybám.

Ako príklad sa často uvádza nízkoúrovňové programovanie v jazykoch ako C, kde síce existuje statický typový systém, ale pomocou pretypovania, ukazovateľov a práce s pamäťou možno typovú kontrolu obísť.

Príklad v C štýle:

```
int x = 42;
char *p = (char *)&x;
```

Tu sa na tú istú pamäť možno pozeráť cez iný typ. Takýto prístup je výkonný, ale nebezpečný.

7.5 Silné a slabé typovanie

Pojmy statické a dynamické typovanie hovoria o tom, *kedy* sa typy kontrolujú. Pojmy silné a slabé typovanie hovoria skôr o tom, *ako prísne* jazyk bráni miešaniu nekompatibilných typov.

- Silne typovaný jazyk nedovolí ľubovoľne miešať hodnoty rôznych typov bez explicitnej konverzie.
- Slabo typovaný jazyk často vykonáva implicitné konverzie.

Príklad slabšieho typovania:

```
"5" + 1
```

V niektorých jazykoch môže byť výsledok "51", inde chyba. To závisí od konkrétnej sémantiky jazyka.

7.6 Rozdiel: statické verzus dynamické typovanie

Vlastnosť	Staticky typovaný jazyk	Dynamicky typovaný jazyk
Kedy sa kontrolujú typy	Pred spustením programu.	Počas behu programu.
Typ majú hlavne	Premenné a výrazy.	Hodnoty počas behu.
Chyby	Vela chýb odhalí kompilátor.	Niektoré chyby sa prejavajú až pri vykonaní danej časti programu.
Flexibilita	Mensia, ale bezpečnejšia.	Väčšia, ale vyžaduje viac testovania.
Príklady	Haskell, Java, Rust.	Racket, Python, JavaScript.

7.7 Sémantika príkazu if

Príkaz alebo výraz `if` môže mať v rôznych jazykoch odlišnú sémantiku. Rozdiely sa týkajú najmä:

- typu podmienky,
- toho, ktoré hodnoty sa považujú za pravdivé,
- typov vetiev `then` a `else`,
- toho, či je `if` výraz alebo príkaz.

7.8 if v staticky typovanom jazyku

V staticky typovanom jazyku je často požiadavka, aby podmienka mala typ `Bool`. Napríklad v Haskell:

```
if x > 0 then "kladne" else "nekladne"
```

Podmienka:

```
x > 0
```

má typ `Bool`.

V Haskellu sú obe vetvy výrazu `if` povinné rovnakého typu:

```
if x > 0 then "kladne" else "nekladne"  
-- typ String
```

Ale výraz:

```
if x > 0 then "kladne" else 0
```

je typová chyba, pretože prvá vetva má typ `String` a druhá typ `Num` a =>
a.

7.9 if v dynamicky typovanom jazyku

V Rackete má `if` tvar:

```
(if podmienka true-vyraz false-vyraz)
```

Materiál uvádza, že `true-vyraz` sa vráti, ak je podmienka rôzna od `#f`; za pravdivé sa považuje aj 0, prázdny zoznam alebo reťazec. Vetvy `true-vyraz` a `false-vyraz` nemusia mať rovnaký typ, pretože Racket je dynamicky typovaný.

Príklad:

```
(define (test x)
  (if (> x 10) "Hura" -7))
```

Volania:

```
(test 20) -- "Hura"
(test 0)  -- -7
```

Výsledok tej istej funkcie teda môže byť raz reťazec a inokedy číslo.

7.10 Pravdivostné hodnoty

Rôzne jazyky majú rôzne pravidlá pravdivosti.

Jazyk	Sémantika podmienky
Haskell	Podmienka musí mať typ <code>Bool</code> .
Racket	Iba <code>#f</code> je nepravda; všetko ostatné sa správa ako pravda.
Python	Existujú <code>truthy/falsy</code> hodnoty: napr. 0, [], "" sú nepravdivé.
C	Nula je nepravda, nenulová hodnota je pravda.

Dôležité je, že nejde iba o typovanie, ale aj o konkrétnu sémantiku jazyka.

7.11 if ako výraz a if ako príkaz

V niektorých jazykoch je `if` výraz, teda vracia hodnotu. V iných jazykoch je `if` príkaz, ktorý iba riadi tok programu.

V Haskellí je `if` výraz:

```
abs x = if x < 0 then -x else x
```

V Rackete je `if` tiež výraz:

```
(if (> x 10) "Hura" -7)
```

V jazykoch ako C alebo Java je `if` primárne príkaz:

```

if (x > 10) {
    y = 1;
} else {
    y = 2;
}

```

Hodnotový výraz sa tam zvyčajne píše pomocou ternárneho operátora:

```
y = x > 10 ? 1 : 2;
```

7.12 Sémantika zoznamov

Aj pojem *zoznam* môže mať v rôznych jazykoch odlišnú sémantiku. Rozdiely sú najmä:

- či zoznam musí byť homogénny,
- či je zoznam definovaný ako algebraický dátový typ alebo ako reťaz dvojíc,
- či môže vzniknúť nesprávny zoznam,
- či sa typ prvkov kontroluje staticky alebo až za behu.

7.13 Zoznamy v Haskell

V Haskell sú zoznamy homogénne. To znamená, že všetky prvky zoznamu musia mať rovnaký typ.

```

[1,2,3]      :: [Int]
["a","b"]    :: [String]
[True,False] :: [Bool]

```

Toto nie je platný zoznam:

```
[1, "ahoj", True]
```

pretože prvky majú rôzne typy.
Zoznam je buď:

```
[]
```

alebo:

```
x : xs
```

kde $x :: a$ a $xs :: [a]$.

7.14 Zoznamy v Rackete

V Rackete sa zoznamy reprezentujú pomocou konštrukcie `cons`. Materiál porovnáva syntax Haskellu a Racketu takto:

Operácia	Haskell	Racket
Prázdny zoznam	<code>[]</code>	<code>null</code>
Konštruktor	<code>(:)</code>	<code>cons</code>
Prvý prvok	<code>head</code>	<code>car</code>
Zvyšok zoznamu	<code>tail</code>	<code>cdr</code>
Test prázdnoti	<code>null</code>	<code>null?</code>

Príklad:

```
(cons 1 (cons 2 null))
```

je to isté ako:

```
(list 1 2)
```

alebo:

```
'(1 2)
```

7.15 Proper a improper list

V Rackete `cons` v skutočnosti vytvára dvojicu, teda *cons cell*. Zoznam je špeciálny prípad vnorených dvojíc, ktoré končia hodnotou `null`.

Správny zoznam:

```
(cons 1 (cons 2 null))  
-- '(1 2)
```

Nesprávny zoznam:

```
(cons 1 2)  
-- '(1 . 2)
```

Toto je dvojica, ale nie správny zoznam:

```
(list? (cons 1 2)) -- #f  
(pair? (cons 1 2)) -- #t
```

V Haskellu podobný nesprávny zoznam nevznikne, pretože typ konšuktora `(:)` je:

```
(:) :: a -> [a] -> [a]
```

Druhý argument teda musí byť opäť zoznam.

7.16 Heterogénne zoznamy v dynamicky typovanom jazyku

Dynamicky typované jazyky často umožňujú heterogénne zoznamy:

```
(list 1 "tri" #t 4)
```

Takýto zoznam môže obsahovať číslo, reťazec, boolovskú hodnotu aj funkciu. To je flexibilné, ale znamená to, že funkcie pracujúce so zoznamom musia často robiť runtime kontroly.

Napríklad jednoduchá funkcia `sum` očakávajúca iba čísla zlyhá, ak zoznam obsahuje aj nečíselné prvky. V materiáli funkcia `sum` pre zoznam s hodnotami 1, funkciou `pokus`, reťazcom `"tri"`, hodnotou 4 a vstupom `x` prejde kompiláciou, ale zlyhá za behu.

7.17 Flexibilnejšie spracovanie heterogénnych zoznamov

Dynamické typovanie umožňuje napísať funkciu, ktorá sa podľa typu prvku rozhodne, ako ho spracuje.

Príklad v Rackete:

```
(define (sum xs)
  (cond [(null? xs) 0]
        [(number? (car xs))
         (+ (car xs) (sum (cdr xs)))]
        [(list? (car xs))
         (+ (sum (car xs)) (sum (cdr xs)))]
        [else
         (sum (cdr xs))]))
```

Táto verzia spočíta čísla aj vo vnorených zoznamoch a iné hodnoty ignoruje. Materiál uvádza, že:

```
(sum '(1 (2 "a") #t (3 ( ) )))
```

vráti:

6

V staticky typovanom jazyku by sme podobnú flexibilitu modelovali napríklad pomocou algebraického dátového typu:

```
data Value = VNumber Int
           | VString String
           | VBool Bool
           | VList [Value]
```

Potom by zoznam mal homogénny typ:

```
[Value]
```

ale jednotlivé hodnoty by mohli reprezentovať rôzne prípady.

7.18 Duck typing

Duck typing je štýl dynamického typovania, pri ktorom nie je dôležitý deklarovaný typ objektu, ale to, či objekt podporuje operácie, ktoré sa na ňom používajú.

Názov vychádza z frázy:

Ak to chodí ako kačka a kváka ako kačka, budeme to považovať za kačku.

V programovaní to znamená: ak objekt podporuje metódy alebo operácie, ktoré potrebujeme, môžeme ho použiť bez explicitného dedenia alebo implementácie rozhrania.

Príklad v Python štýle:

```
def print_name(x):
    print(x.name)

class Person:
    name = "Jana"

class Dog:
    name = "Rex"

print_name(Person())
print_name(Dog())
```

Funkcia `print_name` nepotrebuje vedieť, či `x` je `Person` alebo `Dog`. Stačí, že má atribút `name`.

7.19 Duck typing verzus statické rozhrania

V staticky typovanom jazyku by sme často použili rozhranie alebo typovú triedu.

Príklad v Java štýle:

```
interface HasName {
    String getName();
}
```

Funkcia môže prijať iba hodnoty, ktorých typ deklaruje implementáciu rozhrania `HasName`.

V Haskellí by podobnú úlohu mohla plniť typová trieda:

```
class HasName a where
    getName :: a -> String
```

Duck typing je dynamickejší: objekt nemusí vopred deklarovať, že patrí do nejakého rozhrania. Program sa jednoducho pokúsi použiť požadovanú operáciu a ak objekt túto operáciu nepodporuje, vznikne chyba za behu.

7.20 Výhody duck typingu

- vysoká flexibilita,
- menej explicitných deklarácií,
- ľahké písanie generických funkcií,
- vhodné pre skriptovanie a rýchle prototypovanie.

7.21 Nevýhody duck typingu

- chyby sa objavujú až počas behu,
- rozhranie funkcie nemusí byť zrejmé z typovej anotácie,
- pre väčšie systémy môže byť ťažšie udržať konzistenciu,
- vyžaduje dobré testy a dokumentáciu.

7.22 Príklad: duck typing a sčítanie

V dynamickom jazyku môže funkcia vyzeráť napríklad takto:

```
def add_all(xs):  
    result = 0  
    for x in xs:  
        result = result + x  
    return result
```

Táto funkcia bude fungovať pre zoznam čísel. Môže fungovať aj pre iné objekty, ak podporujú operáciu `+` s aktuálnym akumulátorom.

Ak však niektorý prvok operáciu nepodporuje, chyba vznikne až počas behu. Toto je podobné príkladu z Racketu, kde funkcia `sum` prejde kompiláciou, ale zlyhá až pri pokuse sčítať nekompatibilné hodnoty.

7.23 Vzťah k polymorfizmu

Duck typing je forma ad-hoc polymorfizmu v dynamicky typovaných jazykoch. Funkcia sa môže správať polymorfne, pretože funguje s každou hodnotou, ktorá má potrebné operácie.

Staticky typované jazyky riešia podobné situácie pomocou:

- rozhraní,
- dedičnosti,
- generík,
- typových tried,

- traitov.

Rozdiel je v tom, že v staticky typovanom jazyku sa splnenie požadovaného rozhrania väčšinou kontroluje pred spustením programu, zatiaľ čo pri duck typingu sa zistí až pri použití.

7.24 Porovnanie prístupov

Prístup	Kedy sa kontroluje	Charakteristika
Statické typovanie	Pred behom programu	Bezpečnejšie, menej runtime typových chýb, menej flexibilné.
Dynamické typovanie	Počas behu programu	Flexibilnejšie, ale chyby sa môžu objaviť neskôr.
Bez typovej kontroly	Nekontroluje sa spoľahlivo	Nebezpečné, často nízkoúrovňové, možné nedefinované správanie.
Duck typing	Počas použitia operácie	Dôležité je správanie objektu, nie jeho deklarovaný typ.

7.25 Krátke zhrnutie

Staticky typované jazyky kontrolujú typy pred spustením programu, dynamicky typované jazyky až počas behu. Jazyky bez spoľahlivej typovej kontroly umožňujú pracovať s hodnotami spôsobom, ktorý môže viesť k nedefinovanému alebo nebezpečnému správaniu. Sémantika `if` sa môže líšiť v tom, aký typ musí mať podmienka, čo sa považuje za pravdu a či musia mať vetvy rovnaký typ. Zoznamy môžu byť homogénne a staticky kontrolované ako v Haskell, alebo flexibilnejšie a heterogénne ako v dynamických jazykoch. Duck typing znamená, že rozhodujúce nie je meno typu, ale to, či hodnota podporuje požadované operácie.

8 Funkcionálna vs. OOP dekompozícia, mixins, double dispatch, subtyping, generics a polymorfické funkcie

8.1 Dekompozícia programu

Dekompozícia programu znamená rozdelenie programu na menšie časti tak, aby bol program jednoduchší na pochopenie, rozširovanie a údržbu.

Existujú rôzne štýly dekompozície. Dva dôležité sú:

- funkcionálna dekompozícia,
- objektovo-orientovaná dekompozícia.

Tieto dva prístupy sa líšia najmä tým, či program organizujeme primárne podľa operácií, alebo podľa dátových typov/objektov.

8.2 Funkcionálna dekompozícia

Pri funkcionálnej dekompozícii rozdeľujeme program podľa funkcií, teda podľa operácií, ktoré chceme vykonávať nad dátami.

Dáta sú často reprezentované ako algebraické dátové typy a funkcie potom spracúvajú jednotlivé prípady pomocou pattern matchingu.

Príklad v Haskell:

```
data Expr
  = Lit Int
  | Add Expr Expr
  | Mul Expr Expr
```

Funkcia na vyhodnotenie výrazu:

```
eval :: Expr -> Int
eval (Lit n)      = n
eval (Add x y)    = eval x + eval y
eval (Mul x y)    = eval x * eval y
```

Funkcia na pekný výpis výrazu:

```
pretty :: Expr -> String
pretty (Lit n)      = show n
pretty (Add x y)    = "(" ++ pretty x ++ " + " ++ pretty y ++ ")"
pretty (Mul x y)    = "(" ++ pretty x ++ " * " ++ pretty y ++ ")"
```

Tu je program organizovaný podľa operácií:

- eval,
- pretty,
- prípadne simplify,
- typeCheck,
- compile.

Každá funkcia rieši všetky konštruktory dátového typu.

8.3 Výhody funkcionálnej dekompozície

Funkcionálna dekompozícia je výhodná, keď máme stabilné dátové typy, ale často pridávame nové operácie.

Ak chceme pridať novú operáciu, napríklad zjednodušovanie výrazov, stačí napísať novú funkciu:

```

simplify :: Expr -> Expr
simplify (Add x (Lit 0)) = simplify x
simplify (Add (Lit 0) y) = simplify y
simplify (Mul x (Lit 1)) = simplify x
simplify (Mul (Lit 1) y) = simplify y
simplify (Add x y)      = Add (simplify x) (simplify y)
simplify (Mul x y)      = Mul (simplify x) (simplify y)
simplify e              = e

```

Nemusíme meniť existujúce funkcie.

8.4 Nevýhody funkcionálnej dekompozície

Nevýhoda sa ukáže, keď chceme pridať nový variant dátového typu.

Napríklad pridáme odčítanie:

```

data Expr
  = Lit Int
  | Add Expr Expr
  | Mul Expr Expr
  | Sub Expr Expr

```

Teraz musíme upraviť všetky existujúce funkcie:

- eval,
- pretty,
- simplify,
- všetky ďalšie funkcie nad Expr.

Funkcionálna dekompozícia teda uľahčuje pridávanie nových operácií, ale sťažuje pridávanie nových variantov dát.

8.5 OOP dekompozícia

Pri objektovo-orientovanej dekompozícii rozdeľujeme program podľa objektov alebo tried. Každý typ dát obsahuje operácie, ktoré sa naň vzťahujú.

Príklad v Java štýle:

```

interface Expr {
    int eval();
    String pretty();
}

class Lit implements Expr {
    private final int n;

```

```

    Lit(int n) {
        this.n = n;
    }

    public int eval() {
        return n;
    }

    public String pretty() {
        return Integer.toString(n);
    }
}

class Add implements Expr {
    private final Expr left;
    private final Expr right;

    Add(Expr left, Expr right) {
        this.left = left;
        this.right = right;
    }

    public int eval() {
        return left.eval() + right.eval();
    }

    public String pretty() {
        return "(" + left.pretty() + " + " + right.pretty() + ")";
    }
}

```

Tu je program organizovaný podľa variantov dát:

- trieda `Lit`,
- trieda `Add`,
- trieda `Mul`,
- prípadne ďalšie triedy.

Každá trieda obsahuje implementácie operácií, ktoré sa jej týkajú.

8.6 Výhody OOP dekompozície

OOP dekompozícia je výhodná, keď často pridávame nové dátové varianty, ale množina operácií je stabilná.

Ak chceme pridať nový typ výrazu, napríklad `Sub`, stačí pridať novú triedu:

```

class Sub implements Expr {
    private final Expr left;
    private final Expr right;

    Sub(Expr left, Expr right) {
        this.left = left;
        this.right = right;
    }

    public int eval() {
        return left.eval() - right.eval();
    }

    public String pretty() {
        return "(" + left.pretty() + " - " + right.pretty() + ")";
    }
}

```

Existujúce triedy Lit, Add, Mul netreba meniť.

8.7 Nevýhody OOP dekompozície

Nevýhoda sa ukáže, keď chceme pridať novú operáciu.

Napríklad chceme pridať operáciu:

simplify()

Potom musíme:

- upraviť rozhranie Expr,
- doplniť metódu do každej existujúcej triedy,
- zmeniť všetky implementácie, ktoré triedy realizujú.

OOP dekompozícia teda uľahčuje pridávanie nových variantov dát, ale sťažuje pridávanie nových operácií.

8.8 Expression problem

Rozdiel medzi funkcionálnou a OOP dekompozíciou sa často označuje ako *expression problem*.

Ide o otázku, ako navrhnuť program tak, aby bolo možné jednoducho a typovo bezpečne pridávať:

- nové dátové varianty,
- nové operácie,

bez úprav existujúceho kódu.

Funkcionálny štýl prirodzene podporuje pridávanie nových operácií. Objektový štýl prirodzene podporuje pridávanie nových dátových variantov.

8.9 Porovnanie funkcionálnej a OOP dekompozície

Vlastnosť	Funkcionálna dekompozícia	OOP dekompozícia
Primárne členenie	Podľa operácií.	Podľa dátových typov/tried.
Dáta	Často algebraické dátové typy.	Objekty a triedy.
Spracovanie prípadov	Pattern matching.	Dynamické volanie metód.
Ľahké pridanie	Novej operácie.	Nového typu objektu.
Ťažké pridanie	Nového variantu dát.	Novej operácie.
Typický jazyk	Haskell, ML, OCaml.	Java, C++, C#, Smalltalk.

8.10 Subtyping

Subtyping znamená vzťah medzi typmi, kde hodnotu jedného typu možno použiť tam, kde sa očakáva hodnota iného typu.

Ak platí, že typ S je podtyp typu T , zapisujeme:

$$S <: T$$

a znamená to, že každú hodnotu typu S možno použiť ako hodnotu typu T .

Príklad:

```
class Animal {  
    void speak() { ... }  
}  
  
class Dog extends Animal {  
    void fetch() { ... }  
}
```

Ak `Dog` dedí z `Animal`, potom `Dog` je podtyp `Animal`. Preto môžeme písať:

```
Animal a = new Dog();
```

Premenná typu `Animal` môže obsahovať objekt typu `Dog`.

8.11 Subtypový polymorfizmus

Subtypový polymorfizmus znamená, že rovnaký kód môže pracovať s hodnotami rôznych podtypov spoločného nadtypu.

Príklad:

```
void makeSpeak(Animal a) {
    a.speak();
}
```

Funkcia `makeSpeak` môže dostať objekt typu `Dog`, `Cat`, `Cow`, ak sú všetky podtypmi `Animal`.

Dôležité je, že metóda, ktorá sa zavolá, sa zvyčajne vyberá dynamicky podľa skutočného typu objektu. Tomu sa hovorí dynamický dispatch.

8.12 Dynamický dispatch

Dynamický dispatch znamená, že implementácia metódy sa vyberie až počas behu programu podľa skutočného typu prijímateľa správy.

Príklad:

```
Animal a = new Dog();
a.speak();
```

Hoci premenná `a` má statický typ `Animal`, objekt má dynamický typ `Dog`. Preto sa zavolá implementácia `speak` z triedy `Dog`.

V bežnom OOP je dispatch často jednoduchý, teda podľa jedného objektu: podľa prijímateľa metódy.

8.13 Double dispatch

Double dispatch je technika, pri ktorej sa výsledná operácia vyberá podľa dynamických typov dvoch objektov, nie iba jedného.

Problém nastáva napríklad pri kolíziách objektov v hre:

```
asteroid.collideWith(spaceship)
asteroid.collideWith(bullet)
spaceship.collideWith(asteroid)
```

Chceme, aby sa správanie vybralo podľa oboch typov:

- asteroid – loď,
- asteroid – strela,
- loď – bonus,
- strela – nepriateľ.

Bežný OOP dispatch vyberá metódu podľa prvého objektu. Double dispatch doplní druhý krok, aby sa rozhodlo aj podľa druhého objektu.

8.13.1 Príklad double dispatch

```
interface Shape {
    void intersect(Shape other);

    void intersectWithCircle(Circle c);
    void intersectWithRectangle(Rectangle r);
}

class Circle implements Shape {
    public void intersect(Shape other) {
        other.intersectWithCircle(this);
    }

    public void intersectWithCircle(Circle c) {
        System.out.println("circle-circle");
    }

    public void intersectWithRectangle(Rectangle r) {
        System.out.println("rectangle-circle");
    }
}

class Rectangle implements Shape {
    public void intersect(Shape other) {
        other.intersectWithRectangle(this);
    }

    public void intersectWithCircle(Circle c) {
        System.out.println("circle-rectangle");
    }

    public void intersectWithRectangle(Rectangle r) {
        System.out.println("rectangle-rectangle");
    }
}
```

Volanie:

```
s1.intersect(s2)
```

najprv dynamicky vyberie metódu podľa typu `s1`. Potom sa vnútri zavolá metóda na `s2`, čím sa dynamicky vyberie aj podľa typu `s2`.

Double dispatch je základná myšlienka návrhového vzoru Visitor. Visitor umožňuje definovať novú operáciu nad objektovou štruktúrou bez zmeny tried prvkov tejto štruktúry.

8.14 Mixins

Mixin je znovupoužiteľný kus funkcionality, ktorý možno pridať k triede bez toho, aby išlo o klasickú samostatnú triedu v hierarchii dedičnosti.

Mixin typicky:

- pridáva metódy,
- očakáva, že trieda má určité existujúce metódy alebo vlastnosti,
- slúži na zdieľanie správania medzi rôznymi triedami,
- obmedzuje potrebu viacnásobného dedenia.

Príklad v pseudokóde:

```
mixin JsonSerializable {
    String toJson() {
        return serializeFields(this);
    }
}

class User with JsonSerializable {
    String name;
    int age;
}
```

Trieda `User` získa metódu `toJson`, hoci mixin nie je bežným predkom v zmysle klasickej hierarchie *is-a*.

8.15 Mixin verzus dedičnosť

Klasická dedičnosť vyjadruje vzťah *is-a*:

`Dog extends Animal`

Pes je zviera.

Mixin skôr vyjadruje znovupoužiteľnú schopnosť alebo vlastnosť:

```
User with JsonSerializable
Logger with Timestamped
Button with Clickable
```

Teda nejde vždy o prirodzený vzťah *je typom*, ale skôr o pridanie správania.

8.16 Problém viacnásobného dedenia

Mixiny často riešia problém, ktorý vzniká pri viacnásobnom dedení.

Ak trieda dedí z viacerých tried, môže vzniknúť konflikt:

```
class A {  
    void f() { ... }  
}  
  
class B {  
    void f() { ... }  
}  
  
class C extends A, B {  
    ...  
}
```

Ktorú metódu `f` má trieda `C` zdediť?

Mixiny sa snažia poskytovať znovupoužiteľnosť podobnú viacnásobnému dedeniu, ale s explicitnejšími pravidlami skladania.

8.17 Generics

Generics umožňujú definovať typy a funkcie parametrizované typmi.

Príklad v Jave:

```
class Box<T> {  
    private T value;  
  
    Box(T value) {  
        this.value = value;  
    }  
  
    T get() {  
        return value;  
    }  
}
```

Použitie:

```
Box<Integer> bi = new Box<Integer>(42);  
Box<String> bs = new Box<String>("ahoj");
```

Trieda `Box<T>` je všeobecná. Konkrétny typ vznikne až dosadením typu za parameter `T`.

8.18 Generics a parametrický polymorfizmus

Generics sú príkladom parametrického polymorfizmu. Pri parametrickom polymorfizme definujeme funkciu alebo dátový typ pomocou typových premenných namiesto konkrétnych typov.

V materiáli je parametrický polymorfizmus definovaný tak, že pri definícii funkcie možno použiť typové premenné namiesto konkrétnych typov a jednotlivé inštancie zdieľajú rovnakú implementáciu.

Príklad v Haskellí:

```
id :: a -> a
id x = x
```

Táto funkcia funguje pre každý typ `a`.

Ďalší príklad:

```
map :: (a -> b) -> [a] -> [b]
map _ [] = []
map f (x:xs) = f x : map f xs
```

Funkcia `map` je polymorfná, pretože funguje pre ľubovoľné typy `a` a `b`. Tento typ a implementácia sú uvedené aj v materiáli k parametrickému polymorfizmu.

8.19 Polymorfické dátové typy

Polymorfický môže byť aj dátový typ. Napríklad:

```
data Maybe a = Nothing | Just a
```

Typ `Maybe a` reprezentuje hodnotu, ktorá buď chýba, alebo obsahuje hodnotu typu `a`. Materiál uvádza, že ak je dátový typ parametrizovaný typovým parametrom, hovoríme, že je polymorfný.

Konkrétne typy sú napríklad:

```
Maybe Int
Maybe String
Maybe Bool
```

Samotné `Maybe` je typový konštruktor:

```
Maybe :: * -> *
```

Teda berie konkrétny typ a vytvára nový konkrétny typ.

8.20 Polymorfické funkcie

Polymorfická funkcia je funkcia, ktorá funguje pre viac ako jeden typ.

Príklady:

```
id :: a -> a
id x = x

const :: a -> b -> a
const x _ = x

length :: [a] -> Int
length [] = 0
length (_:xs) = 1 + length xs

map :: (a -> b) -> [a] -> [b]
```

Funkcia `length` je polymorfná, pretože nezávisí od typu prvkov zoznamu. Funguje pre:

```
length [1,2,3]
length ["a","b"]
length [True, False]
```

Vo všetkých prípadoch používa rovnakú implementáciu.

8.21 Ad-hoc polymorfizmus

Ad-hoc polymorfizmus znamená, že jedna operácia môže mať rôzne implementácie pre rôzne typy.

V materiáli je ad-hoc polymorfizmus definovaný ako polymorfizmus, ktorý umožňuje rôzne správanie pre rôzne inštancie polymorfickej funkcie; pre rôzne inštancie typovej premennej existujú rôzne implementácie. Ako príklad sa uvádzajú preťaženie metód v Jave, preťaženie funkcií v C++ a typové triedy v Haskell.

Príklad v Haskell:

```
class Eq a where
    (==) :: a -> a -> Bool
```

Pre rôzne typy môžu existovať rôzne implementácie rovnosti:

```
instance Eq Bool where
    True == True = True
    False == False = True
    _ == _ = False

instance Eq Int where
    x == y = primIntEq x y
```

Funkcia:

```
elem :: Eq a => a -> [a] -> Bool
```

je polymorfná, ale vyžaduje, aby typ `a` podporoval rovnosť.

8.22 Subtyping verus generics

Subtyping a generics sú dva odlišné mechanizmy všeobecnosti.

Subtyping umožňuje písať funkcie nad spoločným nadtypom:

```
void drawAll(List<Shape> shapes) {  
    for (Shape s : shapes) {  
        s.draw();  
    }  
}
```

Funkcia môže pracovať s objektmi typu `Circle`, `Rectangle`, `Triangle`, ak sú podtypmi `Shape`.

Generics umožňujú písať kód parametrický v type:

```
<T> T identity(T x) {  
    return x;  
}
```

Tu nejde o to, že `T` je podtyp nejakého konkrétneho typu. Funkcia funguje pre ľubovoľný typ `T`.

8.23 Bounded generics

Generics možno kombinovať so subtypingom pomocou ohraničení.

Príklad v Jave:

```
<T extends Comparable<T>> T max(T x, T y) {  
    return x.compareTo(y) >= 0 ? x : y;  
}
```

Typový parameter `T` nie je úplne ľubovoľný. Musí byť podtypom `Comparable<T>`. Funkcia teda funguje pre všetky typy, ktoré podporujú porovnávanie.

V Haskellí podobnú úlohu plnia typové triedy:

```
max :: Ord a => a -> a -> a  
max x y = if x >= y then x else y
```

Obmedzenie `Ord a` znamená, že typ `a` musí podporovať usporiadanie.

8.24 Variance pri generics

Pri generických typoch je dôležité, ako sa správa podtypovanie po dosadení typového parametra.

Aj keď platí:

Dog <: Animal

nemusí platiť:

List < Dog > <: List < Animal >

Dôvod je bezpečnosť.

Ak by `List<Dog>` bola podtypom `List<Animal>`, mohli by sme urobiť:

```
List<Dog> dogs = new ArrayList<Dog>();
List<Animal> animals = dogs;
animals.add(new Cat());
```

Teraz by zoznam psov obsahoval mačku. Preto sú generické kontajnery v mnohých jazykoch invariantné, ak jazyk explicitne neumožní kovarianciu alebo kontravarianciu.

8.25 Funkcionálny pohľad na polymorfizmus

Vo funkcionálnom programovaní je polymorfizmus často založený na typových premenných a typových triedach.

Príklad parametrického polymorfizmu:

```
reverse :: [a] -> [a]
```

Funkcia `reverse` nepotrebuje vedieť nič o prvkoch zoznamu. Preto funguje pre zoznam ľubovoľných prvkov.

Príklad ad-hoc polymorfizmu:

```
sort :: Ord a => [a] -> [a]
```

Funkcia `sort` už potrebuje porovnávať prvky. Preto vyžaduje obmedzenie `Ord a`.

8.26 OOP pohľad na polymorfizmus

V OOP sa polymorfizmus často chápe ako možnosť volať rovnakú metódu na objektoch rôznych tried.

Príklad:

```

interface Drawable {
    void draw();
}

class Circle implements Drawable {
    public void draw() { ... }
}

class Rectangle implements Drawable {
    public void draw() { ... }
}

void render(Drawable d) {
    d.draw();
}

```

Funkcia `render` pracuje s každou hodnotou, ktorá implementuje rozhranie `Drawable`. Konkrétna metóda `draw` sa vyberie dynamicky podľa objektu.

8.27 Mixins a polymorfizmus

Mixiny možno chápať ako spôsob horizontálneho skladania správania. Namiesto toho, aby sme všetko organizovali v jednej dedičskej hierarchii, môžeme triede pridať viac nezávislých schopností.

Príklad schopností:

```

Serializable
Comparable
Loggable
Observable

```

Trieda môže získať viacero takýchto vlastností bez toho, aby musela patriť do jednej konkrétnej hierarchie.

Mixiny tak podporujú znovupoužiteľnosť a kompozíciu správania.

8.28 Visitor ako riešenie problému novej operácie v OOP

OOP dekompozícia sťažuje pridávanie nových operácií. Jedným riešením je návrhový vzor `Visitor`.

Namiesto toho, aby sme pridali metódu `pretty`, `eval`, `simplify` priamo do každej triedy, definujeme návštevníka:

```

interface ExprVisitor<R> {
    R visitLit(Lit e);
    R visitAdd(Add e);
    R visitMul(Mul e);
}

```

Každý výraz vie prijať návštevníka:

```
interface Expr {
    <R> R accept(ExprVisitor<R> v);
}
```

Potom novú operáciu reprezentujeme ako novú implementáciu visitoru:

```
class EvalVisitor implements ExprVisitor<Integer> {
    public Integer visitLit(Lit e) { ... }
    public Integer visitAdd(Add e) { ... }
    public Integer visitMul(Mul e) { ... }
}
```

Visitor používa double dispatch: prvý dispatch je volanie `accept` podľa konkrétneho výrazu, druhý dispatch je volanie príslušnej metódy `visit...` podľa konkrétneho typu prvku.

8.29 Porovnanie druhov polymorfizmu

Druh polymorfizmu	Myšlienka	Príklad
Parametrický polymorfizmus	Rovnaká implementácia pre ľubovoľný typ.	<code>id :: a -> a</code> , <code>map :: (a -> b) -> [a] -> [b]</code>
Ad-hoc polymorfizmus	Rôzne implementácie pre rôzne typy.	Typové triedy, preťaženie operátorov.
Subtypový polymorfizmus	Hodnotu podtypu možno použiť tam, kde sa očakáva nadtyp.	<code>Dog</code> ako <code>Animal</code> .
Generics	Typy alebo funkcie parametrizované typmi.	<code>List<T></code> , <code>Box<T></code> .

8.30 Porovnanie mechanizmov

Pojem	Význam
Funkcionálna dekompozícia	Organizácia podľa operácií nad dátami.
OOP dekompozícia	Organizácia podľa objektov/tried, ktoré obsahujú dáta aj metódy.
Mixin	Znovupoužiteľný kus správania prídateľný k triede.
Double dispatch	Výber operácie podľa dynamických typov dvoch objektov.
Subtyping	Možnosť použiť hodnotu podtypu tam, kde sa očakáva nadtyp.
Generics	Parametrizácia funkcií a dátových typov typmi.
Polymorfická funkcia	Funkcia použiteľná pre viacero typov.

8.31 Krátke zhrnutie

Funkcionálna dekompozícia organizuje program podľa operácií nad dátami, zatiaľ čo OOP dekompozícia podľa objektov a tried. Funkcionálny štýl uľahčuje pridávanie nových operácií, OOP štýl uľahčuje pridávanie nových variantov dát. Subtyping umožňuje používať hodnotu podtypu tam, kde sa očakáva nadtyp, a spolu s dynamickým dispatchom tvorí základ OOP polymorfizmu. Double dispatch vyberá operáciu podľa dvoch dynamických typov a používa sa napríklad vo vzore Visitor. Mixiny umožňujú pridávať triedam znovupoužiteľné správanie bez klasickej hierarchie dedičnosti. Generics a polymorfické funkcie umožňujú písať všeobecný kód parametrizovaný typmi.

9 LISP syntax, reťazce a symboly, eval, quasiquote, unquote a string interpolation

9.1 LISP syntax

Jazyky z rodiny LISP, napríklad Common Lisp, Scheme alebo Racket, používajú veľmi jednoduchú a pravidelnú syntax. Program sa skladá zo symbolov, atómov a zoznamov.

V Rackete program pozostáva z termov. Term môže byť napríklad:

- atóm, napríklad `#t`, `#f`, `1`, `2.0`, `null`, `"hello"`, `x`,
- špeciálna forma, napríklad `define`, `lambda`, `if`,
- postupnosť termov v zátvorkách, napríklad `(t1 t2 ... tn)`.

Ak je prvý prvok postupnosti špeciálna forma, výraz má špeciálnu sémantiku. Ak prvý prvok nie je špeciálna forma, postupnosť sa chápe ako volanie funkcie:

`(f a1 a2 ... an)`

Prvý prvok `f` je funkcia a zvyšné prvky sú jej argumenty.

9.2 Prefixový zápis

LISP používa prefixový zápis. Operátor alebo funkcia je vždy na začiatku zoznamu.

Namiesto bežného infixového zápisu:

`1 + 2`

sa píše:

`(+ 1 2)`

Zložitejší výraz:

`(1 + 2) * (3 - 1)`

sa zapíše ako:

`(* (+ 1 2) (- 3 1))`

Výhoda je, že netreba riešiť prioritu operátorov ani asociativitu. Štruktúra programu je priamo daná zátvorkami.

9.3 Úplné zátvorkovanie

V LISP syntaxi sú zátvorky významové, nie iba estetické. Nemožno ich ľubovoľne pridať alebo vynechať.

Napríklad:

`(* (+ 1 2) (- 3 1))`

je správny výraz a výsledok je 6.

Ale:

`(* ((+ 1 2)) (- 3 1))`

je chyba, pretože `(+ 1 2)` sa vyhodnotí na číslo 3 a výraz `(3)` znamená pokus zavolať číslo 3 ako funkciu bez argumentov.

V LISPe teda:

`(3)`

neznamená “číslo 3 v zátvorkách”, ale volanie funkcie 3.

9.4 Program ako dáta

Jedna z najdôležitejších vlastností LISP syntaxe je, že programy majú rovnakú štruktúru ako dátové zoznamy.

Napríklad programový výraz:

`(+ 1 2)`

má tvar zoznamu, ktorého prvý prvok je symbol `+` a ďalšie prvky sú hodnoty 1 a 2.

Tento princíp sa nazýva *homoiconicity*, teda podobnosť alebo zhoda medzi reprezentáciou programu a dát. Vďaka tomu sa v LISPe dobre píšú makrá a programy, ktoré vytvárajú alebo transformujú iné programy.

9.5 Zoznamy

Zoznam v Rackete možno vytvoriť pomocou `list`:

```
(list 1 2 3)
```

alebo pomocou `cons`:

```
(cons 1 (cons 2 (cons 3 null)))
```

Prázdny zoznam je:

```
null
```

Základné operácie nad zoznamami:

Význam	Haskell	Racket/LISP
Prázdny zoznam	<code>[]</code>	<code>null</code>
Konštruktor	<code>(:)</code>	<code>cons</code>
Prvý prvok	<code>head</code>	<code>car</code>
Zvyšok zoznamu	<code>tail</code>	<code>cdr</code>
Test prázdnoty	<code>null</code>	<code>null?</code>

9.6 Reťazce

Reťazec, po anglicky *string*, je postupnosť znakov. V Rackete sa zapisuje v úvodzovkách:

```
"Ahoj"  
"hello world"  
"abc"
```

Reťazce sú dátové hodnoty. Môžeme ich spájať, porovnávať alebo konvertovať.

Príklad:

```
(string-append "a" "b")
```

vráti:

```
"ab"
```

Reťazce porovnávame podľa obsahu napríklad pomocou `equal?`:

```
(equal? "ab" (string-append "a" "b"))  
-- #t
```

Funkcia `eq?` však testuje skôr identitu objektov. Preto môže vrátiť `#f`, aj keď dva reťazce majú rovnaký obsah:

```
(eq? "ab" (string-append "a" "b"))  
-- #f
```

9.7 Symboly

Symbol je atomická hodnota, ktorá sa vypisuje ako identifikátor s apostrofom na začiatku.

Príklad:

```
'a
'hello
'foo
```

Symbol nie je reťazec. Symbol je skôr internované meno alebo identifikátor ako dátová hodnota.

Materiál uvádza:

```
'a
(symbol? 'a)
```

kde:

```
(symbol? 'a)
-- #t
```

Symbody rozlišujú malé a veľké písmená. Teda 'a a 'A sú rôzne symbody.

9.8 Rozdiel medzi reťazcom a symbolom

Reťazec a symbol môžu vyzeráť podobne, ale majú odlišný význam.

Vlastnosť	Reťazec	Symbol
Zápis	"a"	'a
Význam	Postupnosť znakov.	Atomické meno/identifikátor.
Typický účel	Textové dáta.	Označenia, mená, značky.
Porovnanie	Podľa obsahu môže byť lineárne.	Porovnanie býva konštantné.
Konverzia	string->symbol	symbol->string

Konverzie:

```
(string->symbol "a")
-- 'a
```

```
(symbol->string 'a)
-- "a"
```

Materiál uvádza, že:

```
(eq? 'a (string->symbol "a"))
-- #t
```

ale:

```
(eq? (symbol->string 'a) "a")  
-- #f  
  
(equal? (symbol->string 'a) "a")  
-- #t
```

Dôvod je, že `eq?` porovnáva identitu objektov, zatiaľ čo `equal?` porovnáva obsah.

9.9 Porovnávanie: `eq?`, `equal?` a `=`

V Rackete existuje viacero porovnávacích funkcií.

9.9.1 `eq?`

Funkcia `eq?` vracia `#t`, ak dve hodnoty odkazujú na ten istý objekt.

Príklady:

```
(eq? 'a 'a)  
-- #t  
  
(eq? (cons 1 2) (cons 1 2))  
-- #f
```

Dve samostatne vytvorené dvojice majú rovnaký obsah, ale nie sú ten istý objekt.

9.9.2 `equal?`

Funkcia `equal?` porovnáva hodnoty podľa obsahu. Pre páry je porovnanie rekurzívne.

Príklady:

```
(equal? (cons 1 2) (cons 1 2))  
-- #t  
  
(equal? "ab" (string-append "a" "b"))  
-- #t
```

9.9.3 `=`

Funkcia `=` slúži na číselné porovnanie. Ak argumenty nie sú čísla, vznikne chyba.

Príklad:

```
(= 2 2.0)  
-- #t
```


9.10 Quote

Bežný výraz v LISPe sa vyhodnocuje. Napríklad:

```
(+ 1 2)
```

sa vyhodnotí na:

```
3
```

Ak však chceme získať samotný zoznam ako dáta, nie ho vykonať ako program, použijeme `quote`.

```
(quote (+ 1 2))
```

alebo skráteno:

```
'(+ 1 2)
```

Výsledkom je zoznam:

```
(+ 1 2)
```

nie číslo 3.

Materiál uvádza, že:

```
(quote v)
```

je to isté ako:

```
'v
```

`quote` zabráni interpretácii symbolov ako identifikátorov a zoznamov ako volaní funkcií.

Príklady:

```
(quote apple)
```

```
-- 'apple
```

```
(quote 42)
```

```
-- 42
```

```
(quote (1 2 . (3)))
```

```
-- '(1 2 3)
```

9.11 Eval

Funkcia `eval` umožňuje vyhodnotiť dáta ako program.

V jazykoch ako Racket, Scheme, LISP alebo JavaScript možno za behu vytvoriť dáta, ktoré reprezentujú program, a potom ich spustiť. Materiál uvádza príklad v Rackete:

```
(eval '(print "Ahoj"))
```

Tu je:

```
'(print "Ahoj")
```

zoznam ako dátová hodnota. Funkcia `eval` ho interpretuje ako programový výraz a vykoná ho.

V JavaScripte je podobná myšlienka:

```
eval("alert('Ahoj. ')")
```

Rozdiel je v reprezentácii programu:

- v Rackete je program prirodzene reprezentovaný ako vnorený zoznam,
- v JavaScripte býva program často reprezentovaný ako reťazec.

9.12 Prečo je eval silný nástroj

`eval` umožňuje metaprogramovanie, teda programy, ktoré vytvárajú a spúšťajú iné programy.

Možnosti:

- dynamické vytváranie kódu,
- implementácia vlastného jazyka alebo DSL,
- interaktívne prostredia typu REPL,
- experimentovanie s výrazmi vytvorenými za behu.

9.13 Nevýhody a riziká eval

`eval` je silný, ale nebezpečný mechanizmus.

Nevýhody:

- ťažšie sa analyzuje program,
- chyby sa môžu objaviť až za behu,
- zhoršuje optimalizáciu,
- môže predstavovať bezpečnostné riziko, ak sa vyhodnocuje vstup od používateľa,

- runtime prostredie musí obsahovať implementáciu jazyka, napríklad interpreter alebo kompilátor.

Materiál zdôrazňuje, že ak nevieme, aké dáta bude program chcieť za behu zmeniť na program, súčasťou runtime prostredia musí byť aj implementácia jazyka. Tá môže byť vo forme interpretera, kompilátora alebo kombinácie oboch.

9.14 Quasiquote

`quasiquote` je podobné ako `quote`, ale umožňuje do citovaného výrazu vložiť vyhodnotenú časť.

Zápis:

```
(quasiquote v)
```

je skrátené:

```
`v
```

Teda používa sa spätný apostrof, po anglicky backquote.

`quasiquote` sa správa podobne ako `quote`: väčšinu výrazu nechá ako dáta. Rozdiel je v tom, že ak narazí na `unquote`, vyhodnotí príslušný výraz a výsledok vloží do štruktúry.

9.15 Unquote

`unquote` sa používa iba vnútri `quasiquote`. Zápis:

```
(unquote v)
```

je skrátené:

```
,v
```

Význam: výraz `v` sa vyhodnotí a výsledok sa vloží do citovanej štruktúry.

Príklad:

```
`(1 2 ,(+ 1 2) ,(- 5 1))
```

Výsledok je:

```
'(1 2 3 4)
```

Prečo?

- 1 a 2 ostanú ako dáta,
- `,(+ 1 2)` sa vyhodnotí na 3,
- `,(- 5 1)` sa vyhodnotí na 4.

Bez `quasiquote`, teda s obyčajným `quote`, by sa čiarky nevyhodnotili:

```
'(1 2 ,(+ 1 2) ,(- 5 1))
```

Výsledkom by bola štruktúra obsahujúca symboly a zoznamy doslova, nie výsledky výpočtov.

9.16 Použitie quasiquote pri generovaní kódu

`quasiquote` je veľmi užitočný pri generovaní kódu. Umožňuje písať šablónu programu a do nej dopĺňať vypočítané časti.

Príklad:

```
(define x 10)
```

```
‘(+ ,x 5)
```

Výsledok:

```
’(+ 10 5)
```

Tento výsledok je dátová reprezentácia programu. Ak ju odovzdáme do `eval`, môžeme ju vykonať:

```
(eval ‘(+ ,x 5))
```

Výsledok:

15

Teda:

- `quasiquote` vytvorí program ako dáta,
- `unquote` doplní vypočítané hodnoty,
- `eval` program vykoná.

9.17 String interpolation

String interpolation, po slovensky interpolácia reťazcov, je mechanizmus, ktorý umožňuje vložiť hodnotu premennej alebo výsledok výrazu priamo do reťazca.

V JavaScripte:

```
var n = 42;
```

```
‘foo${n}bar‘
```

výsledok je:

```
"foo42bar"
```

Na rozdiel od toho obyčajný reťazec v úvodzovkách interpoláciu nevykoná:

```
"foo${n}bar"
```

výsledok je doslovný reťazec:

```
"foo${n}bar"
```

Materiál uvádza práve tento rozdiel:

```
var n = 42;
```

```
"foo${n}bar" -- "foo${n}bar"
```

```
‘foo${n}bar‘ -- "foo42bar"
```

9.18 Quasiquote verzus string interpolation

`quasiquote` a string interpolation sú podobné v tom, že obe umožňujú vziať šablónu a doplniť do nej vyhodnotené časti.

Rozdiel je v tom, čo sa vytvára:

Mechanizmus	Vytvára	Vkladanie hodnôt
<code>quasiquote</code>	Štruktúrované dáta, často zoznamy alebo kód.	Pomocou <code>unquote</code> , teda <code>,.</code>
String interpolation	Reťazec znakov.	Pomocou špeciálnej syntaxe, napr. <code>\${...}</code> .

Príklad `quasiquote`:

```
'(+ ,x 5)
```

vytvára zoznam:

```
'(+ 10 5)
```

Príklad string interpolation:

```
'x = ${x}'
```

vytvára reťazec:

```
"x = 10"
```

`Quasiquote` teda zachováva štruktúru dát. `String interpolation` vytvára iba text.

9.19 Prečo je `quasiquote` vhodnejší než skladanie reťazcov pri kóde

Ak generujeme program ako reťazec, napríklad:

```
"(" + "+" + " " + x + " 5)"
```

musíme ručne riešiť medzery, úvodzovky, escapovanie a syntaktickú správnosť.

Pri `quasiquote` tvoríme štruktúrované dáta:

```
'(+ ,x 5)
```

Výsledok je priamo zoznam reprezentujúci programový výraz. Je to bezpečnejšie a prirodzenejšie v LISP jazykoch, pretože program a dáta majú podobnú reprezentáciu.

9.20 Malý príklad: tvorba výrazu

Chceme vytvoriť výraz, ktorý sčíta dve čísla:

```
(define (make-add a b)
  '(+ ,a ,b))
```

Potom:

```
(make-add 2 3)
```

vráti:

```
'(+ 2 3)
```

Ak výraz vyhodnotíme:

```
(eval (make-add 2 3))
```

dostaneme:

5

9.21 Krátke zhrnutie

LISP syntax je založená na úplne zátvorkovaných prefixových výrazoch. Programy majú podobnú štruktúru ako dátové zoznamy, čo umožňuje jednoduché metaprogramovanie. Reťazec je postupnosť znakov, zatiaľ čo symbol je atomické meno. `quote` zabráňuje vyhodnoteniu výrazu a umožňuje pracovať s programom ako s dátami. `eval` naopak vezme dáta reprezentujúce program a vykoná ich. `quasiquote` je šablónovanie štruktúrovaných dát: väčšinu výrazu ponechá citovanú, ale časti označené pomocou `unquote` vyhodnotí. String interpolation je podobná myšlienka pre textové reťazce, ale nevytvára štruktúrovaný kód, iba text.

10 Makrá, kedy je ich vhodné použiť, hygienické makrá

10.1 Čo je makro

Makro je jazykový mechanizmus, ktorý umožňuje transformovať zdrojový kód pred jeho vyhodnotením alebo kompiláciou. Makro teda nepracuje primárne s hodnotami počas behu programu, ale so syntaktickou reprezentáciou programu.

V jazykoch z rodiny Lisp, napríklad v Rackete alebo Scheme, majú makrá mimoriadne prirodzené použitie, pretože programy majú podobnú štruktúru ako dátové zoznamy. Program sa dá reprezentovať ako strom alebo vnorený zoznam a makro tento strom transformuje na iný strom.

V Rackete sa program skladá z termov. Term môže byť atóm, špeciálna forma alebo postupnosť termov v zátvorkách. Ak je prvý prvok postupnosti špeciálna forma, výraz má špeciálnu sémantiku; inak ide o volanie funkcie. Makrá umožňujú definovať vlastné špeciálne formy.

10.2 Makrá verzus funkcie

Funkcia dostáva už vyhodnotené argumenty a vracia hodnotu.

Makro dostáva syntaktický tvar výrazu a vytvára nový syntaktický tvar, ktorý sa následne vyhodnotí.

Rozdiel:

Funkcia	Pracuje s hodnotami počas behu programu.
Makro	Pracuje so syntaxou programu pred vyhodnotením.

Tento rozdiel je zásadný. Funkcia nemôže zabrániť vyhodnoteniu svojich argumentov v striktne vyhodnocovanom jazyku, ale makro môže vytvoriť špeciálnu formu, ktorá vyhodnotí iba niektoré časti.

10.3 Prečo nestačí funkcia

Uvažujme príkaz `if`. V striktne vyhodnocovanom jazyku sa argumenty funkcie vyhodnotia pred zavolaním funkcie. Keby bol `if` obyčajná funkcia, vyhodnotili by sa obe vetvy:

```
(my-if condition then-expression else-expression)
```

To je zlé, pretože pri podmienke `True` nechceme vyhodnotiť `else` vetvu a pri podmienke `False` nechceme vyhodnotiť `then` vetvu.

Príklad:

```
(if #t 10 (car null))
```

Tento výraz je v poriadku, pretože `(car null)` sa nevyhodnotí. Ak by však `if` bola obyčajná funkcia, výraz `(car null)` by sa vyhodnotil ešte pred zavolaním funkcie a program by skončil chybou.

Makro dokáže vytvoriť vlastnú riadiacu konštrukciu, ktorá sa správa ako špeciálna forma.

10.4 Makrá v Rackete

V Rackete sa makrá často definujú pomocou:

```
define-syntax  
syntax-rules
```

Príklad makra `my-if`:

```
(define-syntax my-if  
  (syntax-rules (then else)  
    [(my-if p then et else ef) (if p et ef)]  
    [(my-if p then et)          (if p et (void))]))
```

Použitie:

```
(my-if #t then 10 else (car null))
```

Výsledok:

```
10
```

Dôležité je, že `(car null)` sa nevyhodnotí, pretože makro sa rozbalí na skutočný výraz `if`, ktorý vyhodnocuje iba potrebnú vetvu.

Druhé pravidlo umožňuje zápis bez `else` vetvy:

```
(my-if #f then 10)
```

Výsledkom je:

```
#<void>
```

10.5 Syntax makra

Všeobecný tvar:

```
(define-syntax nazov-makra
  (syntax-rules (literal)
    [vzor vysledok]
    ...))
```

Význam:

- `define-syntax` zavádza nové makro,
- `syntax-rules` definuje pravidlá prepisu,
- `literal` sú slová, ktoré sa vo vzore chápu doslovne,
- každé pravidlo má vzor a výsledný tvar.

Pri makre:

```
(define-syntax my-if
  (syntax-rules (then else)
    [(my-if p then et else ef) (if p et ef)]))
```

sú `then` a `else` literály. To znamená, že vo vstupe sa musia objaviť presne tieto symboly.

10.6 Jednoduché makro replace

Príklad:

```
(define-syntax replace
  (syntax-rules ()
    [(replace co cim) cim]))
```

Použitie:

```
(replace (error "Fiha") 7)
```

Výsledok:

7

Prvý argument sa nevyhodnotí. Makro vstupný tvar prepíše iba na druhý argument. To by obyčajná funkcia v striktno vyhodnocovanom jazyku nedokázala, pretože pred jej zavolaním by sa vyhodnotil aj výraz:

```
(error "Fiha")
```

10.7 Makro delay

Makrá sú vhodné aj na odkladanie vyhodnocovania. Príklad z materiálu:

```
(define-syntax delay
  (syntax-rules ()
    [(delay e) (mcons #f (lambda () e))]))
```

Použitie:

```
(delay (begin (print "Pocitam...") 42))
```

Výsledkom nie je okamžité vyhodnotenie výrazu. Výraz sa zabalí do funkcie bez argumentov:

```
(lambda () e)
```

Teda makro `delay` umožní vytvoriť thunk, ktorý možno vyhodnotiť až neskôr.

10.8 Kedy je vhodné použiť makrá

Makrá je vhodné použiť vtedy, keď chceme rozšíriť jazyk o novú syntaktickú konštrukciu alebo špeciálnu formu. Najmä vtedy, keď obyčajná funkcia nestačí.

Typické prípady:

- definovanie nových riadiacich konštrukcií,
- odkladanie vyhodnotenia argumentov,

- vytváranie doménovo špecifických jazykov,
- odstraňovanie opakujúceho sa boilerplate kódu,
- generovanie kódu,
- vytvorenie syntakticky pohodlnejšieho rozhrania.

10.9 Nové riadiace konštrukcie

Makrá sú vhodné napríklad na definovanie vlastnej verzie `if`, `when`, `unless`, `for` alebo `while`.

Príklad:

```
(define-syntax unless
  (syntax-rules ()
    [(unless condition body ...)
     (if condition
         (void)
         (begin body ...))]))
```

Použitie:

```
(unless (= x 0)
  (display "x is not zero")
  (newline))
```

Makro sa môže rozbaľiť na:

```
(if (= x 0)
    (void)
    (begin
      (display "x is not zero")
      (newline)))
```

Takáto konštrukcia sa nedá plnohodnotne nahradiť obyčajnou funkciou, pretože funkcia by v striktno vyhodnocovanom jazyku dostala už vyhodnotené argumenty.

10.10 Odstránenie boilerplate kódu

Makrá sú vhodné aj vtedy, keď sa opakuje rovnaký syntaktický vzor.

Napríklad pri definovaní dátových štruktúr, testov, kontraktov alebo obalových funkcií môže makro vytvoriť viacero definícií naraz.

Príklad v pseudokóde:

```
(define-getters person name age email)
```

môže vygenerovať:

```
(define (person-name p) ...)
(define (person-age p) ...)
(define (person-email p) ...)
```

Takto sa zníži opakovanie a riziko preklepov.

10.11 Doménovo špecifické jazyky

Makrá sú veľmi silné pri tvorbe malých jazykov vložených do hostiteľského jazyka. Takému jazyku sa hovorí DSL, teda *domain-specific language*.

Príkladom môže byť DSL na:

- testovanie,
- zápis gramatík,
- tvorbu HTML,
- definovanie databázových dotazov,
- konfiguráciu systému.

Makrá umožňujú, aby takýto DSL mal vlastnú syntax, ale stále sa prekladal do obyčajného kódu hostiteľského jazyka.

10.12 Kedy makrá nepoužívať

Makrá netreba používať vždy. Ak stačí obyčajná funkcia, je zvyčajne lepšie použiť funkciu.

Makro je menej vhodné, keď:

- nepotrebujeme meniť pravidlá vyhodnocovania,
- nepotrebujeme pracovať so syntaxou,
- stačí obyčajná funkcia vyššieho rádu,
- makro by zhoršilo čitateľnosť,
- makro by skrylo príliš veľa správania,
- vzniknuté chyby by boli ťažko pochopiteľné.

Dobré pravidlo:

Najprv skús použiť funkciu. Makro použi až vtedy, keď funkcia nestačí.

10.13 Problémy s makrami

Makrá sú silné, ale môžu spôsobovať problémy. Typické problémy sú:

- tokenizácia,
- uzátvorkovanie,
- viacnásobné vyhodnotenie argumentov,
- zachytávanie premenných,
- nechcené ovplyvnenie voľných premenných.

10.14 Problém tokenizácie

Pri textových makrách, napríklad v jazyku C, môže makro pracovať priamo so znakmi alebo tokenmi. Ak by pracovalo len so znakmi, mohli by vzniknúť falošné náhrady.

Príklad: ak by sa makro `head` rozbaľovalo na `car`, nechceme, aby sa premenná:

`head-less`

zmenila na:

`car-less`

V jazyku C však výraz:

`head-less`

môže znamenať odčítanie dvoch premenných `head` a `less`. Preto je dôležité, či makrá pracujú so znakmi, tokenmi alebo štruktúrovanou syntaxou.

10.15 Problém uzátvorkovania

V textových makrách môže chýbajúce zátvorkovanie zmeniť význam programu.

Príklad v C:

```
#define MUL(x,y) x*y
```

Použitie:

```
MUL(1+2, 3+4) * 5
```

sa rozbalí na:

```
1+2*3+4*5
```

čo nie je:

`(1+2)*(3+4)*5`

Preto sa C makrá často píše s veľkým množstvom zátvoriek:

```
#define MUL(x,y) ((x)*(y))
```

Racket tento problém nemá v rovnakej podobe, pretože má plne zátvorkovaný syntax. Výraz typu:

```
(macro ...)
```

sa rozbalí na nejaký výraz v tej istej syntaktickej štruktúre.

10.16 Viacnásobné vyhodnotenie argumentov

Textové makrá môžu nechtiac vyhodnotiť argument viackrát.

Príklad v C:

```
#define SQUARE(x) ((x) * (x))
```

Použitie:

```
SQUARE(i++)
```

sa rozbalí na:

```
((i++) * (i++))
```

Premenná `i` sa zvýši dvakrát. To je často neočakávané správanie.

V syntaktických makrách, ako sú makrá v Rackete, sa dá takýmto problémom lepšie predchádzať, ale stále treba rozmýšľať o tom, kôľkokrát sa vložený výraz v rozbalenom kóde použije.

10.17 Hygienické makrá

Hygienické makrá sú makrá, ktoré zabráňujú nechceným konfliktom mien medzi kódom používateľa a kódom vytvoreným makrom.

Makro je hygienické, ak:

- lokálne premenné vytvorené makrom nechtiac nezachytia premenné používateľa,
- voľné premenné v tele makra sa vyhodnocujú v prostredí, kde bolo makro definované, nie v prostredí, kde bolo použité,
- lokálne premenné používateľa môžu korektne zatieniť makrá alebo iné mená podľa pravidiel jazyka.

Cieľom hygieny je, aby rozbalenie makra nemenilo význam programu neočakávaným zachytávaním mien.

10.18 Problém s lokálnymi premennými v makrách

Uvažujme makro:

```
(define-syntax dbl
  (syntax-rules ()
    [(dbl x)
     (let ([y 1])
       (* 2 x y))]))
```

Použitie:

```
(let ([y 7])
  (dbl y))
```

Naivné rozbalenie by mohlo byť:

```
(let ([y 7])
  (let ([y 1])
    (* 2 y y)))
```

Toto je problém, pretože nie je jasné, ktoré `y` má byť použité. Ak by vnútorné `y` z makra zachytilo používateľské `y`, výsledok by bol nesprávny.

Racket tento problém rieši hygienicky: keď je to potrebné, lokálne premenné v makrách nahrádza novými unikátnymi menami. To je jedna časť makro-hygiény.

10.19 Problém s nelokálnymi premennými v makrách

Uvažujme makro:

```
(define-syntax dbl
  (syntax-rules ()
    [(dbl x) (* 2 x)]))
```

Použitie:

```
(let ([* +])
  (dbl 42))
```

Naivné rozbalenie by mohlo byť:

```
(let ([* +])
  (* 2 42))
```

Ak by sa symbol `*` vyhľadával v mieste použitia makra, znamenal by lokálne definované `+`. Výsledok by potom bol:

namiesto očakávaného:

84

Hygienické makrá v Rackete zabezpečia, že * v tele makra sa vyhľadáva lexikálne podľa miesta, kde bolo makro definované, nie podľa miesta, kde bolo použité. To je druhá časť makro-hygiény.

10.20 Lokálne premenné a zatienenie makier

Ďalší problém pri naivných makrách je vzťah medzi lokálnymi premennými a názvami makier.

Ak máme napríklad makro **head**, ktoré sa rozbaľuje na **car**, nechceme, aby lokálna premenná s názvom **head** bola automaticky prepísaná na **car**.

Príklad:

```
(let ([head 1] [car 2])
  head)
```

Naivné rozbalenie by mohlo byť:

```
(let ([car 1] [car 2])
  car)
```

To by spôsobilo duplicitnú definíciu alebo zmenu významu programu.

Racket tento problém nemá: lokálne premenné zatieniajú makrá. V jazyku C sa podobným problémom často predchádza konvenciou, že názvy makier sú veľkými písmenami.

10.21 Makrá v C verzus makrá v Rackete

Makrá v C sú prevažne textové alebo tokenové prepisy vykonávané preprocesorom. Makrá v Rackete pracujú so štruktúrovanou syntaxou jazyka.

Vlastnosť	C makrá	Racket makrá
Úroveň práce	Text/tokeny.	Syntaktické objekty.
Typické problémy	Zátvorkovanie, viacnásobné vyhodnotenie, konflikty mien.	Hygiena veľa konfliktov rieši automaticky.
Vyhodnocovanie	Preprocesor pred kompiláciou.	Rozbaľovanie syntaxe pred behom programu.
Hygiena	Nie je hygienická automaticky.	Štandardne hygienická.

10.22 Makrá a špeciálne formy

Špeciálna forma je syntaktická konštrukcia, ktorá má vlastné pravidlá vyhodnocovania. Napríklad:

```
if  
define  
lambda  
quote
```

nie sú obyčajné funkcie, pretože ich argumenty sa nevyhodnocujú bežným spôsobom.

Makrá umožňujú programátorovi definovať nové konštrukcie, ktoré sa správajú podobne ako vstavané špeciálne formy.

Príklad:

```
(my-if p then e1 else e2)
```

nie je obyčajné volanie funkcie. Je to nový syntaktický tvar, ktorý sa makrom prepíše na vstavané `if`.

10.23 Makrá a fázy spracovania programu

Zjednodušene:

1. Zdrojový kód sa načíta a sparsuje.
2. Makrá sa rozbalia.
3. Výsledný program sa kompiluje alebo interpretuje.
4. Program sa spustí.

Makrá teda patria do fázy pred behom programu. Preto vedia meniť syntax programu, ale nemajú sa zamieňať s obyčajnými runtime funkciami.

10.24 Výhody makier

- umožňujú rozširovať jazyk,
- znižujú opakovanie kódu,
- umožňujú definovať nové riadiace konštrukcie,
- umožňujú odkladať alebo meniť vyhodnocovanie,
- sú vhodné na tvorbu DSL,
- v LISPe/Rackete sú prirodzené vďaka tomu, že program je reprezentovaný ako dátová štruktúra.

10.25 Nevýhody makier

- môžu zhoršiť čitateľnosť programu,
- môžu skryť skutočný vykonávaný kód,
- chyby po rozbalení makra môžu byť ťažšie pochopiteľné,
- makrá sú zložitejšie než obyčajné funkcie,
- pri nehygienických makrách hrozia konflikty mien.

10.26 Praktické pravidlá používania

Makro je vhodné použiť, keď:

- potrebujeme vlastné pravidlá vyhodnocovania,
- potrebujeme pracovať so syntaxou programu,
- chceme zaviesť novú špeciálnu formu,
- obyčajná funkcia by argumenty vyhodnotila príliš skoro,
- chceme odstrániť opakujúci sa syntaktický vzor.

Makro nie je vhodné použiť, keď:

- stačí obyčajná funkcia,
- chceme iba vypočítať hodnotu z hodnôt,
- makro by bolo neprehľadnejšie než explicitný kód,
- makro by zavádzalo neštandardnú syntax bez jasného prínosu.

10.27 Porovnanie pojmov

Pojem	Význam
Makro	Transformácia syntaxe programu pred vyhodnotením.
Funkcia	Výpočet nad hodnotami počas behu programu.
Špeciálna forma	Konštrukcia s vlastnými pravidlami vyhodnocovania.
<code>define-syntax</code>	Zavedenie makra v Rackete.
<code>syntax-rules</code>	Pravidlá prepisu syntaxe makra.
Hygienické makro	Makro, ktoré zabráňuje nechcenému zachytávaniu mien.
Zachytávanie premenných	Situácia, keď makro nechtiac zmení význam premennej.

10.28 Krátke zhrnutie

Makrá sú mechanizmus na transformáciu syntaxe programu pred jeho vyhodnotením. Na rozdiel od funkcií nepracujú s už vypočítanými hodnotami, ale so samotným kódom. Sú vhodné vtedy, keď chceme definovať nové špeciálne formy, zmeniť pravidlá vyhodnocovania, odložiť vyhodnotenie, generovať kód alebo vytvoriť malý doménovo špecifický jazyk. Ak stačí obyčajná funkcia, je lepšie použiť funkciu. Hygienické makrá zabráňujú nechceným konfliktom mien a zachytávaní premenných. Racket má hygienický makrosystém, ktorý rieši problémy typické pre naivné alebo textové makrá, napríklad makrá v jazyku C.